

컴퓨터수학2

20/09/03 ~ 20/12/06

박지원

목차

1. 9-1강_관계와 그들의 특성 이론
2. 9-2강_n항 관계와 응용
3. 9-3강_관계의 표현
4. 9-4강_관계의 폐쇄
5. 9-5강_동치 관계
6. 9-6강_부분 순서
7. 10-0강_그래프에 대한 교수님의 추가 강의
8. 10-1강_그래프 응용 사례
9. 10-2강_그래프 용어와 특별한 그래프 이론
10. 10-3강_그래프 표현방법과 동형 그래프
11. 10-4강_연결성 이론
12. 10-5강_오일러와 해밀턴
13. 10-6강_최단경로
14. 11-1강_트리 소개
15. 11-3강_트리 순회
16. 11-4강_신장 트리
17. 11-5강_최소 신장 트리

<9-1강_관계와 그들의 특성 이론> 9/3 ~ 9/9

1. 사전지식

1) 순서쌍

= 두 집합의 원소들 사이의 관계를 표현하는 가장 직접적인 방법입니다.

= **이진관계**(Binary Relation)랑 동등한 의미입니다.

- [예] $A=\{a, b, c\}$, $B=\{1, 2\}$
- $R \subseteq A \times B = \{(a,1), (a,2), (b,1), (b,2), (c,1), (c,2)\}$

2) 집합의 곱셈 = 데카르트 곱

$A = \{1, 2, 3\}$, $B = \{1, 2\}$ 라고 하면 $A \times B$ 를 구하여라.

SOL

$$A \times B = \{(1,1), (2,1), (3,1), (1,2), (2,2), (3,2)\}$$

집합의 곱셈의 결과는 새로운 집합입니다.

$A \times B$ 의 원소의 개수

= A의 원소의 개수 X B의 원소의 개수

3) 부분 집합

1. 부분집합 개념체크 3, 필수예제 4, 5

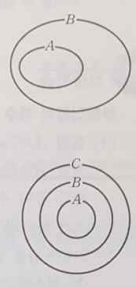
(1) 부분집합
 집합 A 의 모든 원소가 집합 B 에 속할 때, 즉 $x \in A$ 이면 반드시 $x \in B$ 일 때, A 는 B 에 포함된다 또는 B 는 A 를 포함한다고 하며 $A \subset B$ 또는 $B \supset A$ 로 나타낸다. 이때, 집합 A 를 집합 B 의 **부분집합**이라고 한다.

(2) 부분집합의 성질

- ① 임의의 집합은 자기 자신의 부분집합이다.
 즉, 임의의 집합 A 에 대하여 $A \subset A$
- ② 공집합은 모든 집합의 부분집합이다.
 즉, 임의의 집합 A 에 대하여 $\emptyset \subset A$
- ③ $A \subset B$ 이고 $B \subset C$ 이면 $A \subset C$ 이다.

(3) 부분집합의 개수
 집합 $A = \{a_1, a_2, a_3, \dots, a_k, \dots, a_n\} (k \leq n)$ 에 대하여

- ① 집합 A 의 부분집합의 개수: 2^n



4) 관계와 집합

= 관계를 집합과 연관지어서 생각하면 매우 쉽게 이해됩니다. (교집합, 합집합, 차집합)

관계의 결합

A 에서 B 로의 관계는 $A \times B$ 의 부분집합이므로, 두 **집합**이 결합되는 방법에 의해 A 에서 B 로의 두 관계는 결합될 수 있다. 예제 17~19를 보자.

$A = \{1, 2, 3\}$ 이고 $B = \{1, 2, 3, 4\}$ 라 하자. 관계 $R_1 = \{(1, 1), (2, 2), (3, 3)\}$ 과 $R_2 = \{(1, 1), (1, 2), (1, 3), (1, 4)\}$ 는 다음을 구하기 위해 결합될 수 있다.

$R_1 \cup R_2 = \{(1, 1), (1, 2), (1, 3), (1, 4), (2, 2), (3, 3)\}$
 $R_1 \cap R_2 = \{(1, 1)\}$
 $R_1 - R_2 = \{(2, 2), (3, 3)\}$
 $R_2 - R_1 = \{(1, 2), (1, 3), (1, 4)\}$

2. 관계의 정의

$a R b$ = A와 B가 관계가 있다.

$\overline{a R b}$ = A와 B가 관계가 없다.

- 관계 = Relation = R

= 가장 쉽게 이해하는 방법은 관계는 그래프에서 Edge 입니다.

= 여담으로 , 어떤 집합의 원소는 그래프에서 Node(=Vertex) 입니다.

- 집합 A는 컴퓨터학부 학생들의 집합이라 하고 집합 B는 컴퓨터학부 전공과목의 집합이라고 합시다. a는 컴퓨터 학부 학생이고 b는 전공과목 이라고 합시다. 'a는 과목 b를 수강 한다'로 구성되어 있는 관계를 R이라고 합시다. (박지원 , 컴퓨터수학2)는 R에 포함 되어 있습니다. 만약 a는 컴퓨터 학부 학생이며 여자인 학생이고 b는 전공과목인 관계를 R2라 하면 (박지원 , 컴퓨터수학2)는 R2에 포함 되지 않습니다.

- 집합 A에서 집합 B로의 관계는 집합 A X B의 부분집합입니다.

3. 관계의 특성

1) 집합 A에 관한 관계는 A에서 A로의 관계입니다. = 집합 A에 관한 관계는 A X A 의 부분집합입니다. = (A의 원소 , A의 원소)

2) 집합 A에 관한 관계 R이 모든 원소 a E A에 대해 (a,a) E R 이면 **반사(reflexive)**라고 합니다.

3) 집합 A에 관한 관계 R이 모든 a,b E A에 대해 (a,b) E R일 때 (b,a) E R이면 **대칭(symmetric)** 이라고 합니다.

= a 와 b를 바꾸어서 생각해봅니다.

4) 집합 A에 관한 관계 R이 모든 a,b E A에 대해 (a,b) E R일 때 (b,a) E R일 때 a=b이면 **반대칭(antisymmetric)**이라 합니다.

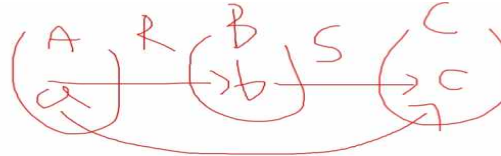
= 관계의 모든 원소가 (a,b)형태로 관계에 존재 할 때 (b,a)가 관계에 존재 하지 않을 때 입니다.

ex) {(2,1) , (3,1) , (3,2) , (4,1) , (4,2) , (4,3)} 에는 (1,2) , (1,3) , (2,3) , (1,4) , (2,4) , (3,4)가 존재하지 않습니다.

5) 집합 A에 관한 관계 R이 모든 a,b,c E A에 대해 (a,b) E R이고 , (b,c) E R일 때 (a,c) E R이면 **전이(transitive)**라고 합니다.

= 삼단 논법 생각하면 됩니다.

6) R을 집합 A에서 집합 B로의 관계라 하고 S를 집합 B에서 C로의 관계라 하자. R과 S의 합성은 순서쌍(a,c)로 구성되는 관계입니다. 여기서 $a \in A$, $c \in C$ 이고 $(a,b) \in R$ 과 $(b,c) \in S$ 인 원소 $b \in B$ 가 존재합니다. R과 S의 합성을 $S \circ R$ 로 표시합니다.(합성함수 개념입니다.)



- 대칭, 반대칭, 전이를 찾을 때는 (a,a)같은 원소는 고려하지 않습니다. 즉, 반사관계는 고려하지 않습니다.
- 어떤 관계가 대칭과 반대칭을 모두 가질 수도 있고, 또는 모두 갖지 않을 수 있습니다.

4. 문제풀이

<예제 6번> n개의 원소를 갖는 집합에는 몇 개의 관계가 있느냐?

= $A \times A$ 의 부분집합의 개수는 2^{n^2} 개입니다.

= 답은 $2^{(n^2)} = 2$ 의 n의 제곱

ex) 집합 {a,b,c}에 존재하는 총 관계의 개수는 몇 개인가 ? $2^{(3^2)} = 2^9 = 512$ 개

<예제 7번>

R1 ~ R6중에 (1,1) (2,2) (3,3) (4,4)의 관계를 모두 포함하고 있어야 반사 관계 이므로 R3,R5가 정답입니다.

<예제 10번>

대칭은 쉽습니다.

R4, R5, R6 가 반대칭입니다.

<예제 13번>

관계의 특성을 이용하면 쉽게 찾을 수 있습니다.

R1 : (3,4) (4,1) → (3,1)이 없습니다.

R2 : (2,1) (1,2) → (2,2)가 없습니다.

R3 : (4,1) (1,2) → (4,2)가 없습니다.

R4 : 전이적입니다.

R5 : 전이적입니다.

R6 : 원소가 하나면 전이적입니다.

<예제 19번>

$R_1 \oplus R_2$ = 합집합에서 교집합을 빼는 연산입니다.

<예제 20번>

합성함수처럼 R의 (1,1) S의(1,0)이 있다면 합성 관계는 (1,0)이 됩니다.

<연습문제 4번 & 12번>

‘a가 b보다 키가 크다’ = 반사 X , 비반사 O , 대칭 X , 반대칭 O , 전이 O

‘a와 b가 같은 날에 태어났다.’ = 반사 O , 비반사 X, 대칭 O , 반대칭 X , 전이 O

‘a와 b는 이름이 같다.’ = 반사 O , 대칭 O , 비반사 X, 반대칭 X , 전이 O

‘a와 b는 같은 조부모를 갖는다.’ = 반사 O , 비반사 X, 대칭 O , 반대칭 X , 전이 X

- ‘a와 b는 같은 조부모를 갖는다.’가 전이가 되지 않는 이유 : a와 b는 친척이고 b와 c가 사촌일 때 a 와 c는 관계가 없을 수 있습니다.

- 비반사 = (a,a)가 R에 단 한 개도 존재 하지 않습니다. a는 a보다 키가 크다가 절대 불가능 합니다.

<연습문제 26번>

- 역관계 = 뒤집은 관계
- 보수 관계 =

<연습문제 42번>

예제 6번과 동일한 문제입니다. 2개의 원소를 갖는 집합에는 몇 개의 관계가 있는가?

$$2 ^ { (2 \times 2) } = 16\text{개}$$

<9-2강_n항 관계와 응용> 9/17 ~ 9/20

1. n항 관계

= 셋 이상의 집합의 원소간의 관계

- 이진 관계에서 집합이 추가된 것이 전부입니다.
- 이진 관계의 필기를 보고 확장해서 공부하는 것이 좋습니다.
- 부분집합 빈칸 뚫어서 무조건 시험에 나옵니다.

• 정의 1. $A_1, A_2, \dots, A_n$ 을 집합이라 하자.
이 집합들에 대한 n항 관계는 $A_1 \times A_2 \times \dots \times A_n$ 의 부분집합이다.

• $A_1, A_2, \dots, A_n$ 을 관계의 정의역 (domain)이라 부르고,

• n을 차수 (degree)라 부른다.

2. Key(데이터베이스 개념)

1) Primary Key(기본키)

- = Candidate Key 중에서 DB 설계자가 정한 가장 대표적인 키.
- 반드시 1개입니다.
- 여러 속성으로 구성 될 수 있습니다.
- 유일성과 최소성을 모두 만족시킵니다.

2) Candidate Key(후보키)

- = 기본키가 될 수 있는 조건을 만족한 키.
- 여러 개일 수 있으며 테이블에 반드시 1개는 있어야 합니다.
- 유일성과 최소성을 모두 만족시킵니다.

3) Alternative Key(대체키 , 보조키)

- = 후보키 중에서 기본키가 아닌 모든 키.
- 유일성과 최소성을 모두 만족시킵니다.

4) Super Key(슈퍼키)

- = 하나 이상의 속성들을 조합하여 유일성을 만족시키는 키.
- 유일성은 만족시키지만 최소성은 만족시키지 않습니다.
- Relation DB Model 내부에서 보통 매우 많이 존재합니다.

5) Composite Key(합성키)

- = 두 개 이상의 속성으로 이루어진 키.
- = 학생 테이블에서 Major와 학점 각각은 Super Key가 될 수 없습니다. 하지만 둘의 데카르트 곱을 하여 둘을 합친다면 Super Key가 될 수 있습니다.

3. Projection(투사)

- = 몇 개를 골라냅니다.
- = n-tuple에서 m-tuple로의 mapping 입니다.
- <예제10> 번을 보면 이해가 바로 됩니다.

- 표 3의 관계에 투사 $P_{1,2}$ 가 적용되면 어떤 테이블이 얻어나?

TABLE 3 Enrollments.		
<i>Student</i>	<i>Major</i>	<i>Course</i>
Glauser	Biology	BI 290
Glauser	Biology	MS 475
Glauser	Biology	PY 410
Marcus	Mathematics	MS 511
Marcus	Mathematics	MS 603
Marcus	Mathematics	CS 322
Miller	Computer Science	MS 575
Miller	Computer Science	CS 455

TABLE 4 Majors.	
<i>Student</i>	<i>Major</i>
Glauser	Biology
Marcus	Mathematics
Miller	Computer Science

- $P_{1,2}$ 는 1번 속성과 2번 속성을 뽑아서 Mapping 시키는 것 입니다.

4. Join(결합)

- = 두 테이블에서 중복 되는 속성이 있는 튜플 들을 모두 합칩니다.
- <예제11> 번을 보고 참조 합니다.

5. 문제 풀이

<예제 1>

(1,2,3) 은 R에 포함됩니다.

(2,4,3)은 R에 포함되지 않습니다.

차수는 3

정의역은 $N \times N \times N$

<예제 2 + 연습문제 26>

(1,3,5)은 R에 포함됩니다.

(2,5,9)은 R에 포함되지 않습니다.

차수는 3

정의역은 $Z \times Z \times Z$

(a,b,c)에서 a를 1로 고정했을 때 (1,2,3) , (1,3,5) , (1,4,7)처럼 다양하게 존재하여 어느 것 하나 원소를 특정하게 정할 수 없으므로 primary key (기본키)가 없습니다.

<예제 7>

Key의 조건을 보고 해당 관련된 Tuple을 고르면 됩니다.

<예제 12 , 13>

SQL 명령어로 테이블을 보는 법.

<연습문제 6>

- primary key는 Course_number입니다.
- Professor와 Department를 합쳐도 합성키가 불가능합니다.
- Professor와 Room을 합쳐도 합성키가 불가능 합니다.
- Professor와 Time을 합치면 합성키가 가능 합니다.
- Professor와 Course_number을 합치면 합성키가 가능 합니다.

<연습문제 8>

- primary key는 ISBN으로 적합합니다.
- 합성키가 어떤 조건에서 되는지는 간단하게 생각하면 쉽습니다. = 겹치지 않으면 됩니다.

<연습문제 12>

project = 2 , Quantity >= 50을 만족하는 것은 9191 Part_number입니다.

<연습문제 20 ~ 23>

연습문제 12와 같이 보면 , project = 2가 C1 , Quantity >= 50을 C2라고 생각합시다.

- $s_{C_1 \wedge C_2}(R) = s_{C_1}(s_{C_2}(R))$
- $s_{C_1}(s_{C_2}(R)) = s_{C_2}(s_{C_1}(R))$
- $s_C(R \cup S) = s_C(R) \cup s_C(S)$
- $s_C(R \cap S) = s_C(R) \cap s_C(S)$

다음의 4가지 식은 모두 당연합니다.

<연습문제 25>

= 관계 R과 S를 합집합 하고 투사한 것과 관계 R을 투사하고 관계 S를 투사한 것의 합집합은 같습니다.

ex) R = {(a,b)} , S = {(a,c)} 이면 R 합집합 S 는 {(a,b) ,(a,c)}입니다. 이것의 투사는 {a}입니다.

P1(R) = {a} , P1(S) = {a}이므로 두 투사의 합집합은 {a}이므로 같습니다.

<연습문제 26>

= 관계 R과 S를 교집합 하고 투사한 것과 관계 R을 투사하고 관계 S를 투사한 것의 교집합은 다릅니다.

ex) $R = \{(a,b)\}$, $S = \{(a,c)\}$ 이면 R 교집합 S 는 공집합입니다. 공집합의 투사는 공집합입니다. $P1(R) = \{a\}$, $P1(S) = \{a\}$ 입니다.

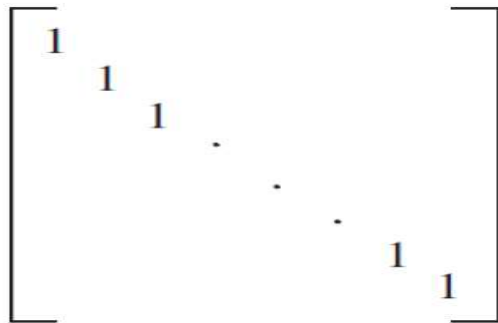
<9-3강_관계의 표현> 9/26

- 관계는 0-1행렬 또는 방향성 그래프로 관계를 표기합니다.
- 집합과 관계를 코딩테스트에서 자주 풀던 그래프와 연결 짓고 나서 매우 쉬워졌습니다.
- 컴퓨터 입장에서는 0-1행렬 으로 관계를 이해하는 게 쉽습니다.
- 인간 입장에서는 방향성 그래프로 관계를 이해하는 것이 더 직관적입니다.

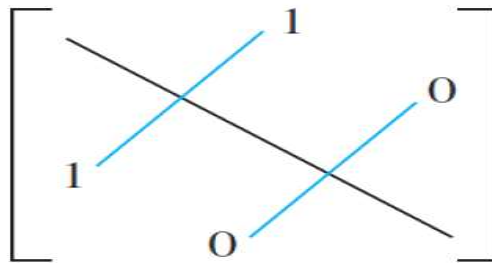
1. 행렬의 특성

= 반사, 대칭, 반대칭

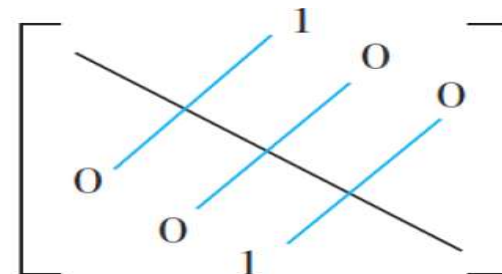
- 관계를 표현하는 행렬 1개를 위해서는 집합 2개가 필요합니다.
- 반사는 좌상단 -> 우하단의 대각선이 모두 1이어야 합니다. 비반사는 모두 0 입니다.
- 대칭은 자신과 자신의 transpose가 같을 때를 의미합니다.
- 반대칭은 대칭이 1개도 없어야 합니다. (0,0)은 포함해도 괜찮습니다.



반사



대칭



반대칭

2. 관계의 합성과 부울 곱

1) 관계의 합성

$$\mathbf{M}_{S \circ R} = \mathbf{M}_R \odot \mathbf{M}_S$$

$$A \odot B = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \odot \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} (1 \wedge 0) \vee (0 \wedge 1) & (1 \wedge 0) \vee (0 \wedge 1) \\ (0 \wedge 0) \vee (1 \wedge 1) & (0 \wedge 0) \vee (1 \wedge 1) \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}$$

= 관계 S와 관계 R을 합성한 관계의 행렬은 관계 R 행렬과 관계 S 행렬의 합성입니다.

- 부울 곱으로 증명할 수 있습니다.

2) 부울 곱

$$c_{ij} = (a_{i1} \wedge b_{1j}) \vee (a_{i2} \wedge b_{2j}) \vee \cdots \vee (a_{ik} \wedge b_{kj}).$$

위의 식은 시험에 나옵니다.

- Tip : 곱하기 에서 한 개라도 둘 다 1인 것은 1이고 그 외에는 모두 0입니다. -> 부울 곱 정의로 이해할 수 있습니다.
- $A \odot B$ 에서 첫 번째 계산은 $(1,0) \odot (0,1)$ 입니다. 다시 말해 $(1 \wedge 0)$, $(0 \wedge 1)$ 입니다. $(1 \wedge 1)$ 꼴이 단 한 개도 없으므로 0 입니다.
- $A \odot B$ 에서 세 번째 계산은 $(0,0) \odot (1,1)$ 입니다. 다시 말해 $(0 \wedge 0)$, $(1 \wedge 1)$ 입니다. $(1 \wedge 1)$ 꼴이 한 개 존재하므로 1 입니다.

<Example>

공식(Tip)을 모르고 계산하면 아래와 같이 귀찮습니다.

$$\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

$$\mathbf{A} \odot \mathbf{B} = \begin{bmatrix} (1 \wedge 1) \vee (0 \wedge 0) & (1 \wedge 1) \vee (0 \wedge 1) & (1 \wedge 0) \vee (0 \wedge 1) \\ (0 \wedge 1) \vee (1 \wedge 0) & (0 \wedge 1) \vee (1 \wedge 1) & (0 \wedge 0) \vee (1 \wedge 1) \\ (1 \wedge 1) \vee (0 \wedge 0) & (1 \wedge 1) \vee (0 \wedge 1) & (1 \wedge 0) \vee (0 \wedge 1) \end{bmatrix}$$

3. Directed Graph(방향성 그래프)

= vertex(정점)의 집합 V와 edge(모서리)의 집합 E로 구성되어 있습니다.

= 정점 a는 모서리 (a,b)의 시작점이고, 정점 b는 모서리의 끝점입니다.

- 관계는 반드시 방향성 그래프이며, 비방향성 그래프는 관계라고 칭하지 않습니다.

- 반사, 대칭, 반대칭 인지 찾기는 쉬우나 전이는 찾기가 어렵습니다.

- 반사는 루프를 찾으면 됩니다.

- Vertex a 와 vertex b가 있을 때 a -> b 인 edge 와 b -> a 인 edge가 존재하면 대칭입니다.

4. 문제풀이

<예제1>

A의 원소 a가 B의 원소 b보다 클 때 1 , 작거나 같을 때는 0

<예제2>

행렬을 보고 관계를 도출할 수 도 있습니다. 원소가 1(True)인 항목만 뽑아서 순서쌍을 만들어 주셨습니다.

<예제3>

반사적이고 , 대칭이고 , 반대칭은 아닙니다.

- 대칭과 반대칭은 공존할 수 없습니다.

<예제4>

r1과 r2의 합집합 = 둘 중 하나라도 1이면 1 입니다.

r1과 r2의 교집합 = 둘 다 1이면 1입니다.

<예제5>= 부울 곱의 정의를 통해 쉽게 계산할 수 있습니다.

<예제7>

= (a,b) , (a,d) , (b,b) , (b,d) , (c,a) , (c,b) , (d,b)는 연결되어 있는 순서쌍이므로 연결해줍니다.

= (b,b)같은 경우는 루프 입니다.

<예제9>

= 그래프와 순서쌍의 개념을 이해하면 됩니다.

<예제10>

= 대칭이기만 합니다. 나머지 3개는 아닙니다.

<연습문제8>

- 반사는 b , 비반사는 c , 대칭은 a , c , 반대칭은 b

<연습문제10>

- a) : $(1000+1) * 500$ 개
- b) : $1 + 998 * 2 + 1$ 개
- c) : $1 \leq a \leq 999$ 인 개수이므로 999개
- d) : $1 \sim 1000 + 1 \sim 1000$ 의 모든 경우에서 합이 1001보다 작은 개수는 500500개.
- e) : 모든 원소 입니다. 100만개

<연습문제12>

= 역관계에 대한 행렬은 역행렬과 다릅니다.

= 역관계에 대한 행렬은 그 행렬의 Transpose입니다.

<연습문제16>

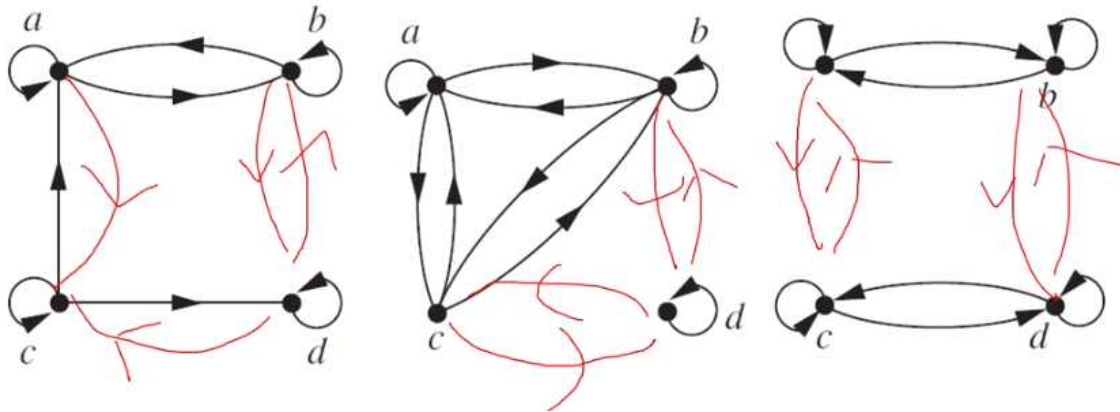
= 연습문제 12와 같습니다. 역관계에 대한 행렬과 행렬의 transpose에 0이 아닌 항목의 개수는 같습니다.

<연습문제32>

= 전이 말고는 그래프에서 반사 , 대칭 찾는 법을 보고 풀면 됩니다.

<연습문제36>

= 보수 관계는 연결 되지 않은 부분을 연결하면 됩니다.



- 붉은색이 보수 관계 입니다.

<9-4강_관계의 폐쇄> 9/28

1. 관계의 폐쇄를 만족하기 위한 정의

= 관계 S(반사 폐쇄, 대칭 폐쇄, 전의 폐쇄)는 특성 P(반사, 대칭, 전이) 관계 R의 폐쇄(Closure)

조건 1 : S는 P를 갖고 R을 포함해야 합니다.

조건 2 : S는 R을 포함하는 P를 갖는 모든 관계의 부분 집합입니다.

1. Reflexive Closure(반사 폐쇄)

집합 $A = \{1, 2, 3\}$ 에 관한 관계 R

$R = \{(1, 1), (1, 2), (2, 1), (3, 2)\}$, $(2, 2), (3, 3)$

– 반사적인가?

- 관계 R에 관계 R에는 존재하지 않는 (2,2) 과 (3,3)의 반사 관계를 추가하여 반사 폐쇄를 이루었습니다.
- 반사 폐쇄는 폐쇄의 조건 1, 조건 2를 만족하는 관계 중에 크기가 가장 작아야 합니다.
- 즉 반사 폐쇄는 하나만 존재 합니다.

– $R \cup \Delta$

– $\Delta = \{(a, a) \mid a \in A\} \rightarrow$ 대각 관계 (diagonal relation)

2. Symmetric Closure(대칭 폐쇄)

- 관계 R에 관계 R에는 존재하지 않는 대칭 관계를 추가하여 대칭 폐쇄를 이루었습니다.

ex) 관계 R에 (1,2)가 존재하고 (2,1)은 존재하지 않을 때 (2,1)을 추가하여 대칭 폐쇄를 이룹니다.

- $R \cup R^{-1}$

3. 전이 폐쇄와 방향성 그래프

- 주먹 구구 식으로 찾는 것이 매우 복잡하고, 실수할 가능성이 매우 높으므로 알고리즘으로 해결해야 합니다.

ex) 관계 R이 (1,3), (1,4), (2,1), (3,2) 일 때 이 원소들만의 전이 폐쇄를 이루기 위해서 (1,2), (2,3), (2,4), (3,1)을 추가합니다. 그 결과 원소가 8개가 된 관계 R은 전이폐쇄를 만족한 것 같지만 사실은 그렇지 않습니다. 새로 생긴 (3,1) 과 기존의 (1,4) 사이에서 새로운 전이 관계인 (3,4)가 필요하기 때문입니다. 그러므로 주먹 구구 식으로 찾다 보면 전이 폐쇄를 찾는 것이 매우 어렵습니다.

1) 중요 개념_1번

정의 2. R을 집합 A에 관한 관계라 하자. 연결 관계 (connectivity relation) R^* 는 R에 a에서 b로의 적어도 길이 1 이상의 경로가 있는 쌍 (a, b)로 구성된다.

정리 1. R을 집합 A에 관한 관계라 하자. n이 양의 정수일 때 a에서 b로의 길이 n의 경로가 있다는 것과 $(a, b) \in R^n$ 는 동등하다.

- R^n 은 정점 a에서 정점 b로 갈 때 n개의 edge를 지나서 가는 방법입니다. 부울 곱으로 R^n 을 구할 수 있습니다.

ex) R^3 은 a에서 b를 갈 때 3개의 edge(2개의 vertex)를 지나서 도착하는 경로입니다. = $a \rightarrow d \rightarrow c \rightarrow b$

ex) R^1 은 a에서 b로 갈 때 다른 vertex를 거치지 않고 바로 가는 경로입니다.

- R^* 은 a에서 b로 갈 때 1 ~ n개의 edge를 지나서 도착하는 모든 경로의 합집합입니다. 그러므로 최소 1개의 edge는 지나므로 위의 정의를 만족합니다. 다시 말해, R^1 도 R^* 에 포함되고, R^2 도 R^* 에 포함되고, R^{107} 도 R^* 에 포함됩니다. a로부터 b로의 경로가 존재하는 순서쌍(a,b)로 구성되는 관계라고도 합니다.

ex) $R^* = R_1 \cup R_2 \cup R_3 \dots \cup R_n$ // n 은 집합 A 의 원소의 개수 , 즉 Vertex의 개수입니다.

- $(a,b) \in R$, $(b,c) \in R$, $(c,d) \in R$ 이라면 $(a,d) \in R^3$ 입니다.
- $(a,c) \in R$ 이고 $(c,b) \in R^n$ 이면 $(a,b) \in R^{n+1}$ 입니다.
- 위의 예시들에서 R 은 R_1 과 같습니다.
- 교재에서는 3개의 edge를 지나서 도착하는 경로를 거리가 3인 경로라고 표현하는데 , 후에 가중치가 있는 edge의 경로에서는 이 개념이 맞지 않다고 생각이 들어서 해당 정의를 적지 않았습니다.

2) 중요개념_2번

정리 2. 관계 R 의 전이 폐쇄는 연결관계 R^* 와 같다.

- $(a,b) \in R^*$ 이고 $(b,c) \in R^*$ 이면 $(a,c) \in R^*$ 입니다.
- 교수님 증명을 이해하지 못했으나 일단 정리 2를 외웁시다.

3) 중요개념_3번

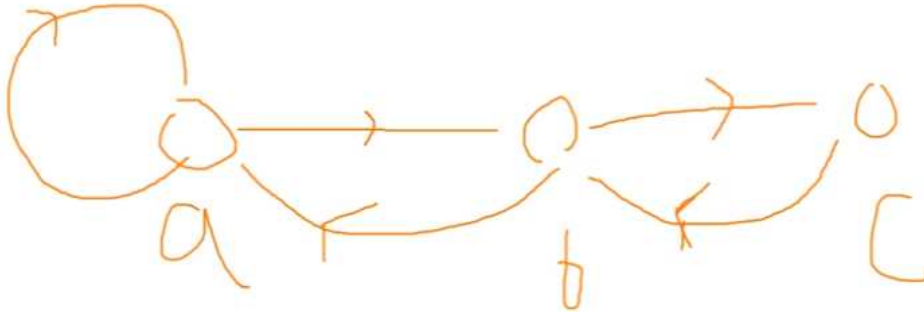
보조정리 1. A 를 n 개의 원소를 갖는 집합이라 하고, R 을 A 에 관한 관계라 하자.

R 에 a 에서 b 로 적어도 길이 1인 경로가 있으면 길이 n 을 초과하지 않는 경로가 있다.

더욱이 $a \neq b$ 일 때 a 에서 b 로 적어도 길이 1인 경로가 있으면 길이 $n - 1$ 을 초과하지 않는 경로가 있다.

A 의 정점이 3개일 때 A 의 어떤 점에서 4개의 edge를 지나서 A 의 어떤 점을 가려면 반드시 중복된 Vertex가 발생합니다.

= **비둘기 집의 원리**입니다.(비둘기가 4마리고 집이 3마리면 반드시 어떤 집은 2마리 이상 들어갑니다.)



ex) (a,a) 의 R_4 = a 에서 4개의 edge를 지나 a 로 가는 경로 = $a \rightarrow b \rightarrow c \rightarrow b \rightarrow a$ 이므로 중복된 b 가 존재합니다.

성질 P에 대한 관계 R의 폐쇄

- R을 포함하고, 성질 P를 갖고, R을 포함하면서 성질 P를 갖는 어떤 관계 안에 포함되는 것이 존재한다면 이런 관계 S

5. 문제풀이

<예제1>

기초 반사폐쇄 문제.

<예제2>

기초 대칭폐쇄 문제.

<예제5>

R_n 은 몇 개의 지하철을 타서 다른 역으로 가는 걸 의미합니다.

(고속터미널 , 송실대) = 7호선 , (사평 , 고속터미널) = 6호선 $\rightarrow$ (고속터미널 , 송실대) = 7호선 + 6호선

<예제6>

R^* 을 하면 R 을 많이 하는 것 이므로 접경하지 않은 지역들을 갈수 있는 방법이 됩니다.

<예제7>

- 주어진 MR 이 3x3행렬이므로 , 관계의 정점은 3개입니다. 그러므로 n 은 3이 됩니다.
- MR_1 은 주어진 MR 이고 MR_2 , MR_3 를 부울 곱으로 계산합니다.
- $R^* = R_1 \cup R_2 \cup R_3 \cup \dots \cup R_n$ 의 정의에 의해 결과로 나온 MR_1 , MR_2 , MR_3 를 합집합 합니다.
- $\bigvee$ 는 합집합 기호입니다.

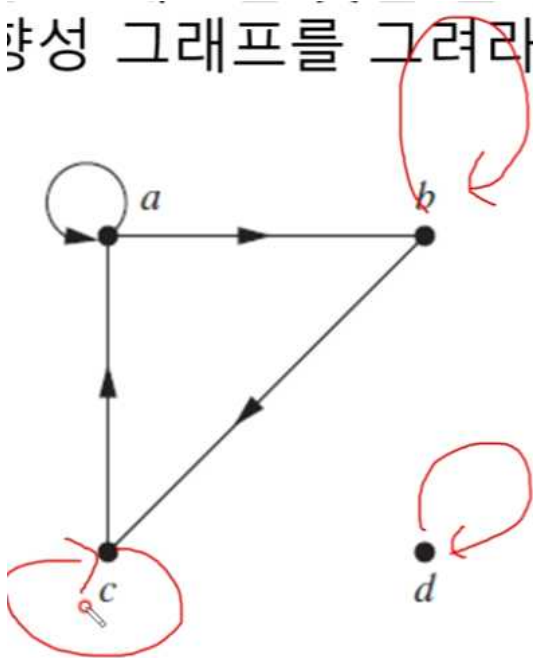
$$M_{R^*} = M_R \bigvee M_R^{[2]} \bigvee M_R^{[3]} .$$

<연습문제2>

- 반사 폐쇄를 만족 시키려면 $a = b$ 를 모두 포함시킵니다.

<연습문제6>

양성 그래프를 그려라.



<연습문제12>

= 반사폐쇄를 행렬로 구현하기

= MR(원래의 행렬) U 단위행렬

<단위행렬>

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

<연습문제13>

= 대칭폐쇄를 행렬로 구현하기

= MR(원래의 행렬) U MR-1(Transpose)

<연습문제26>

= Vertex는 5개 이고 , 부울 곱을 이용합니다.

a) A2를 부울 곱으로 구하고 A3도 부울 곱으로 구합니다. 그런데 A4와 A5를 부울 곱으로 구하다보니 A2와 A4가 같고 A3와 A5가 같습니다.

그러므로 $A^* = A \cup A2 \cup A3 \cup A2 \cup A5$

b) a와 같습니다.

<연습문제35> 매우 중요한 문제 !!!

= R은 이미 반사적이므로 반사 폐쇄는 존재하지 않습니다.

<9-5강_동치 관계> 10/3 ~ 10/6

1. 동치(Equivalent Relation)

= 어떤 관계가 반사, 대칭, 전이를 모두 성립할 때 동치라고 합니다.

= $A \sim B$ 로 표기합니다.

❖ 예제 : 집합 $A=\{1,2,3,4\}$ 에 대한 다음 관계가 동치관계인지 판별하여라.

(1) $R_1=\{(1,1), (1,4), (2,2), (2,3), (3,2), (3,3), (4,1), (4,4)\}$

(2) $R_2=\{(1,1), (1,2), (1,3), (1,4), (2,2), (2,3), (2,4), (3,3), (3,4), (4,4)\}$

<풀이>

(1) 관계 R_1 은 동치관계다.

(2) 관계 R_2 은 대칭관계가 성립하지 않으므로 동치관계가 아니다.

2. 동치류(Equivalence Class)

= 집합 A에 대하여 동치관계인 R이 있다고 가정합시다. 집합 A의 각 원소 x에 대해 $[x]$ 를 R에 대한 x의 동치류라고 정의 합니다.

$$\blacklozenge [x] = \{y \mid (x, y) \in R\}$$

(2) 집합 $\{1,2,3,4,5\}$ 에 대한 관계 $R_2 = \{(1,1), (1,3), (1,5), (2,2), (3,1), (3,3), (3,5), (4,4), (5,1), (5,3), (5,5)\}$

(2) 반사관계, 대칭관계, 추이관계가 모두 성립하므로 동치관계다.
따라서 동치류는 $[1]=[3]=[5]=\{1,3,5\}$, $[2]=\{2\}$,
 $[4]=\{4\}$ 이다.

- 쉽게 말하면 동치 관계를 이루는 관계의 순서쌍에서 나 말고 다른 원소는 나와 동치류인 원소입니다.

- 위의 예제에서 $(1,1), (1,3), (1,5)$ 은 1은 각각 1, 3, 5와 동치류 입니다.

3. 동치류로 인한 증명(3개가 모두 같은 의미)

$$(i) aRb \quad (ii) [a] = [b] \quad (iii) [a] \cap [b] \neq \emptyset$$

- 관계 R은 동치관계입니다. a와 b는 동치관계인 집합의 원소입니다.
- 관계 R이 동치라면 R에 포함된 원소들은 반사, 대칭, 전이를 만족하는 성질을 이용하여 증명합니다.

1) 1번식 -> 2번식 증명

- $c \in [a]$ 라면 동치류 이므로 관계 R에 (a,c) , (c,a) 가 모두 존재합니다.
- $a R b$ 이므로 관계 R에 (a,b) , (b,a) 가 존재하므로 $b R a$ 입니다.
- (b,a) 가 되고 (a,c) 가 되므로 전이의 성질로 인해 (b,c) 가 가능 하고 (c,b) 도 가능합니다.

2) 2번식 -> 3번식 증명

- 반사의 성질로 $[a]$ 에는 최소 a가 존재하고, $[b]$ 에는 최소 b가 존재합니다.
- 고로 $[a]$ 와 $[b]$ 가 같다면 공집합은 존재할 수 없습니다.

3) 3번식 -> 1번식 증명

- 공집합이 아니므로 (a,c) , (b,c) 가 존재합니다. 그러면 전이와 대칭을 이용해 (a,c) , (c,b) 가 존재하므로 (a,b) 가 존재함을 알 수 있습니다.

4. 분할

= 어떤 집합을 분할하면 분할 된 집합들끼리는 교집합이 없고, 모두 합치면 초기의 전체 집합이 됩니다.

- 분할하여 집합의 크기를 줄입니다.

- 분할의 부분집합들 끼리는 동치류를 이룹니다.

ex) 집합 S가 있을 때 분할한 집합들을 집합 S에 대한 분할 이라고 합니다.

5. 문제풀이

<예제1>

- $a R b$ 는 $a = b$ or $a = -b$ 일 때 반사, 전이, 대칭을 만족하므로 동치입니다.

<예제3>

- 나머지(mod)를 통해 알 수 있는 성질.
- (-7) 을 5로 나눈 나머지는 $= 5 * -2 + 3$ 이니까, 3 입니다. 나머지는 음수가 없습니다.

<예제4>

- 같은 문자열 가능, 대칭인 문자열 가능, a 와 b 의 문자열의 길이가 같고, b 와 c 의 문자열의 길이가 같으면 a 와 c 의 문자열이 같습니다.

<예제6>

- 2 나누기 1은 관계에 포함이 되지만, 1 나누기 2는 관계에 포함되지 않습니다.

<예제7>

- 반사 : $x - x = 0$ 이므로 성립합니다.
- 대칭 : $|x-y| < 1$ 이면 $|y-x| < 1$ 이므로 성립합니다.
- 전이 : (x,y,z) 가 $(0,0.9,1.8)$ 이면 $|x,y| < 1$, $|y,z| < 1$ 이지만 $|x,z|$ 는 1.8이므로 전이를 만족시키지 않습니다.

<예제9>

- 0에 관한 동치류 = $-8, -4, 0, 4, 8 \dots$
- 1에 관한 동치류 = $-3, 1, 5 \dots$

<예제 13>

A3 $\rightarrow R = \{(6,6)\}$

A2 $\rightarrow R = \{(4,4), (4,5), (5,4), (5,5)\}$

A1 $\rightarrow R = \{(1,1), (2,2), (3,3), (1,2), (2,1), (1,3), (3,1), (2,3), (3,2)\}$

<예제 14>

[0] = -8, -4, 0, 4, 8 ...

[1] = -7, -3, 1, 5, 9 ...

[2] = -6, -2, 2, 6, 10 ...

[3] = -5, -1, 3, 7, 11 ...

- 이제 모든 정수를 4개의 동치류로 구분할 수 있습니다. [0], [1], [2], [3] 끼리는 교집합이 없지만 합집합은 정수 전체 입니다.

<연습문제2>

a) 동치입니다.

b) 동치입니다.

c) 아빠만 같거나 엄마만 같은 경우 이므로 동치가 아닙니다.

d) 전이 X $\rightarrow$ a와 b가 만나고 b와 c가 만나다고 해서 a와 c가 만난다고 확신할 수 없습니다.

e) c와 동일한 방법으로 동치가 아닙니다.

<연습문제10>

- $f(x) = [x]_{\mathbb{R}}$ 입니다.

$$(i) aRb \quad (ii) [a] = [b] \quad (iii) [a] \cap [b] \neq \emptyset$$

<연습문제16>

$a \sim f$ 는 양의 정수입니다.

- 반사O : $((a,a), (a,a))$ 가 R에 속합니다.
- 대칭O : $((a,b), (c,d))$ 가 R에 속하면 $((c,d), (a,b))$ 가 R에 속합니다.
- 전이X : $((a,b), (c,d))$ 가 R에 속하면 $((c,d), (e,f))$ 가 R에 속합니다.
- $ad = bc$ 이고 $cf = de$ 일 때 $af = be$ 를 증명하면 됩니다. $ad*cf = bc*de$ 에서 양변에 c 와 d 를 나누면 $af = be$ 가 나옵니다.
그러므로 동치입니다.

<연습문제22>

- 그래프의 동치관계입니다.

<연습문제24>

- 행렬의 동치관계입니다.
- a)는 대칭이 되지 않습니다.
- b)는 동치가 됩니다.
- c)는 동치가 됩니다.
- 어떻게 교수님은 전이를 이렇게 빨리 찾을까?

<연습문제36>

- a) 4를 2로 나눈 것과 나머지가 같은 원소입니다. = -4, -2, 0, 2, 4
- a) 4를 3로 나눈 것과 나머지가 같은 원소입니다. = -5, -2, 1
- a) 4를 6로 나눈 것과 나머지가 같은 원소입니다. = -8, -2, 4
- a) 4를 8로 나눈 것과 나머지가 같은 원소입니다. = -4, 4, 12

<연습문제40>

- (1,2)의 동치류는 $(c, 2c)$ 이므로 (1,2) , (2,4) ,(3,6) ...입니다.
- $c * (1,2)$ 로 표현할 수 있습니다.

<연습문제42>

- 분할은 서로 교집합이 없어야 합니다.
 - 분할된 집합을 모두 합치면 원래의 집합이 나와야 합니다.
- = a) ,c) 가 정답입니다.

<연습문제44>

- 1) X : 음수가 없습니다.
- 2) X : 0이 없습니다.
- 3) O
- 4) O
- 5) X : 교집합 되는 것이 많습니다.

<연습문제48>

- a) $(a,a),(a,b),(b,a),(b,b) // (c,c),(d,d),(c,d),(d,c) // (e,e),(f,f),(g,g),(e,f),(e,g),(f,e),(g,e),(f,g),(g,f)$
- b) ~ d)도 동등한 원리로 합니다.
 - b)의 {a} 같은 경우는 (a,a)가 나올 수 있습니다.

<연습문제54>

= 어려운 증명

<연습문제62>

= 동치관계의 개수 세기 문제는 가능한 분할의 개수를 세는 문제입니다.

- 4개의 원소가 하나의 집합 = 1개
- 1개 / 3개 = 4개
- 2개 / 2개 = 3개
- 2개 / 1개 / 1개 = 6개
- 1개 / 1개 / 1개 / 1개 = 1개
- 순서는 상관 없습니다. $\rightarrow \{a\} / \{b, c, d\} = \{a\} / \{c, d, b\}$

<연습문제64>

= 전이 상태의 관계에 반사 폐쇄를 한 후에 대칭 폐쇄를 하면 언뜻 보기에는 동치를 만족할 것 같지만 그렇지 않습니다.

- $\{(1, 2), (3, 2)\}$
 - 반사 폐쇄
 - 대칭 폐쇄
- $(1, 1), (2, 2), (3, 3)$
 $(2, 3), (2, 1)$

<9-6강_부분순서> 10/7 ~ 10/13

*** 이 때 부터 관계라는 걸 그래프의 Vertex(=Node) 와 Edge와 같은 개념으로 이해를 하고나서 필기의 정확성과 엄밀성이 높아졌습니다. ***

1. Partial Order(부분 순서)

= 관계 R이 **반사적이고** , **반대칭적이고** , **전이적** 이면 관계 R을 Partial Order 하다고 부릅니다.

= 간단하게 생각해 보면 노드와 노드의 관계로 이루어진 **그래프에서 노드의 값에 순서(대소 , 사전식 순서)가 존재하는 것** 입니다. 순서를 생각해 보면 반사 , 반대칭 , 전이가 모두 이해가 됩니다.

ex) 반사 : $1 \geq 1$, 반대칭 : $2 \geq 1$ 라면 $1 > 2$ 는 불가능 , 전이 : $3 \geq 2$ & $2 \geq 1 \rightarrow 3 \geq 1$ 은 자명 합니다.

<사전식 순서>

= 데카르트 곱 OR 문자열의 부분 순서

$(a1 , a2) < (b1 , b2)$ 을 성립하는 2가지 경우

1) $a1 < b1$

2) $a1 = b1$ & $a2 < b2$

2. Linear Order(선형 순서)

= Total Order(전체 순서)라고도 합니다.

= 집합 S에서 모든 원소 두 원소가 **비교가 가능**하다면 S를 Linear Order라고 합니다.

$$a \preceq b$$

$$a < b$$

같은 걸 포함하거나 , 포함 하지 않거나로 구분 됩니다.

3. 순서화집합

정의 4. $(S, \leq)$ 가 $\leq$ 이 전체순서이고 모든 공집합이 아닌 S 의 부분집합이 최소원소를 가지는 부분집합이면 순서화집합이다.

- 최소원소는 집합에서 가장 왼쪽에 있는 원소.

ex) 사전에서 맨 앞 단어

= 예제8의 예시로 이해 합시다.

정리 1. S 를 순서화 집합이라 하자.

귀납단계: 모든 $y \in S$ 에 대해, $x < y$ 인 모든 $x \in S$ 에 대해 $P(x)$ 가 참이면 $P(y)$ 가 참이다. 그러면, 모든 $x \in S$ 에 대해, $P(x)$ 가 참이다.

= 증명하셨는데 이해 못함

= 이해를 제대로 못했으므로 다시 들읍시다.

4. Hasse Diagram(하세도표)

1) 정의

= 부분순서 그래프를 반사(루프)와 , 전이를 제거하고 모서리의 방향을 표현하지 않은 간단한 그래프

2) 특성

= 특성들을 합쳐서 덮음관계 라고 합니다.

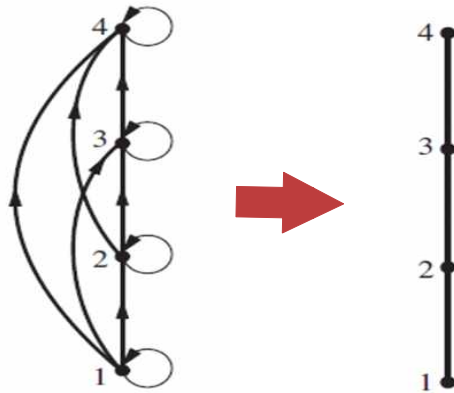
① 부분순서는 정해진 순서가 존재하므로 한 방향으로만(작은 노드 -> 큰 노드) Edge가 존재하므로 화살표를 생략합니다.

② 부분순서는 반사의 특성을 갖고 반사는 Loop를 의미하므로 Loop을 생략합니다.

③ 부분순서는 전이의 특성을 갖고 전이는 가장 짧은 경로들로 만들 수 있으므로 생략합니다.

<Example>

- 화살표를 삭제
- Loop을 삭제
- (a,c)는 (a,b) & (b,c)로 그릴 수 있으므로 모두 삭제 //ex) (2,4) -> (2,3) & (3,4)



3) 극대와 극소

① 극대

= 어떤 그래프에서 Node_A를 고르고 A와 연결된(관계가 있는) 다른 모든 Node들과 비교했을 때 Node_A가 모두 큰 경우 Node_A를 극대원소 라고 합니다.

= 하세도표에서 보면 자신에게서 나가는 Edge만 있고 들어오는 Edge는 없는 경우.

② 극소

= 어떤 그래프에서 Node_A를 고르고 A와 연결된(관계가 있는) 다른 모든 Node들과 비교했을 때 Node_A가 모두 작은 경우 Node_A를 극소원소 라고 합니다.

= 하세도표에서 보면 자신에게 들어오는 Edge만 있고 나가는 Edge는 없는 경우

- 극대 , 극소는 각각 1개 이상일 수 있습니다.

- 예제14를 참고합니다.

4) 최대와 최소

① 최대

= 극대가 그래프에서 오직 1개만 존재할 때 최대 입니다.

② 최소

= 극소가 그래프에서 오직 1개만 존재할 때 최소 입니다.

5) 상한계와 하한계

① 상한계

= 해당 부분집합의 모든 원소보다 어떠한 경우에도 크거나 같은 원소.

② 하한계

= 해당 부분집합의 모든 원소보다 어떠한 경우에도 작거나 같은 원소.

- 상한계 , 하한계 모두 반드시 같은 경우도 포함시켜야 합니다.

③ 최소상한계

= 상한계 중에서 가장 작은 원소 1개

④ 최대하한계

= 하한계 중에서 가장 큰 원소 1개

ex) 상한계의 원소가 d, e, f 일 때 f 는 d 와 e 보다 크지만 d 와 e 는 크기를 비교할 수 없을 때 최소상한계는 존재하지 않습니다.

6) 격자

= 노드들 중에서 2개의 노드를 뽑았을 때 언제나 최소상한계와 최대하한계가 모두 존재할 때 그 그래프를 격자라고 합니다.

- 예제 21

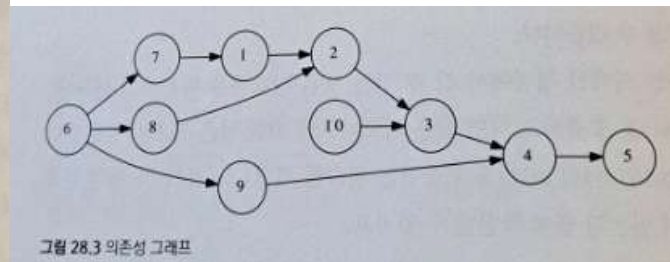
5. Topological Sort

= 위상 정렬

= 그래프가 부분순서 관계 일 때 노드들끼리 순서가 존재해서 노드들을 해당 순서로 정렬하는 방법입니다.

- 공집합이 아닌 집합에서 관계가 부분순서일 때 그래프를 만들고, **그래프에서 극소원소를 선택하고 제외하는 행동을 반복**하여 구합니다. 극소원소는 1개 이상 일 수 있으므로, 그런 상황에서 어떤 극소를 뽑는 것은 프로그래머의 자유이므로 **위상정렬은 다양하게 나올 수 있습니다.**

1. 김치를 다진 마늘과 볶는다.
2. 볶은 김치에 돼지고기, 고춧가루, 다진 마늘을 넣고 더 볶는다.
3. 멸치 육수를 붓고 끓인다.
4. 파와 양파를 넣고 좀더 끓인다.
5. 맛있게 먹는다.
6. 냉장고에서 재료들을 꺼낸다.
7. 김치를 썰다.
8. 돼지고기를 썰다.
9. 파와 양파를 썰다.
10. 멸치 육수를 낸다.



<Example>

부분순서집합 $(\{1, 2, 4, 5, 12, 20\}, |)$ 에 대해 양립할 수 있는 전체순서를 찾아라.

- '|' = 나누어 떨어집니다.
- 1 -> 5 -> 2 -> 4 -> 20 -> 12

6. 문제풀이

<예제1>

반사됨, 반대칭됨, 전이 됨

$a \geq a \rightarrow a \geq a$, $a \geq b \rightarrow b \geq a$ 불가능, $a \geq b$, $b \geq c \rightarrow a \geq c$

<예제2>

부분순서임

$a \% a = 0$, $9 \% 3 = 0 \rightarrow 3 \% 9 \neq 0$, $9 \% 3 = 0$, $27 \% 3 = 0 \rightarrow 27 \% 9 = 0$

<예제3>

멱집합 = 모든 집합의 부분 집합.

ex) $\{a, b\}$ 의 멱집합 = 공집합, $\{a\}$, $\{b\}$, $\{a, b\}$

$\{a\}$ 는 $\{a\}$ 에 포함, $\{a\}$ 가 $\{a, b\}$ 에 포함 $\rightarrow \{a, b\}$ 가 $\{a\}$ 에 안포함, $\{a\}$ 가 $\{a, b\}$ 에 포함 $\{a, b\}$ 가 $\{a, b\}$ 에 포함 전이

<예제4>

반사가 안됨.

<예제5>

3과 9는 비교 가능함.

5와 7은 비교 불가능 $\rightarrow$ 서로소

<예제6>

정수 집합에서 집합의 원소 a , b 에서 $a \leq b$ 이거나 $b \leq a$ 를 만족하므로 전체 순서

<예제7>

양의 정수 집합에서 집합의 원소들끼리 나누어 떨어지는 관계를 만들 때 5와 7은 나누어 떨어지지 않으므로 비교가 불가능합니다. 그러므로 전체 순서 X

<예제8>

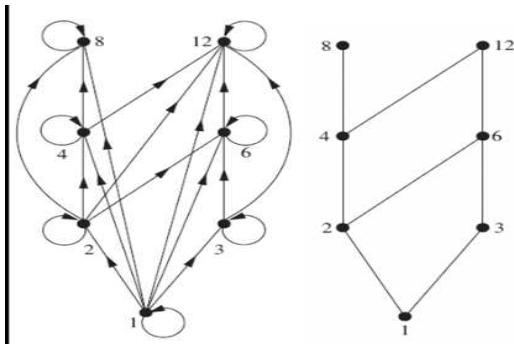
순서화집합 문제.

어떤 순서쌍 집합이 (양의정수, 양의정수) 이라고 가정하고 $\{(1,1), (1,2), (2,1)\}$
 $(1,1) \leq (1,2), (1,2) \leq (2,1)$ 이고 최소원소가 $(1,1)$ 임.

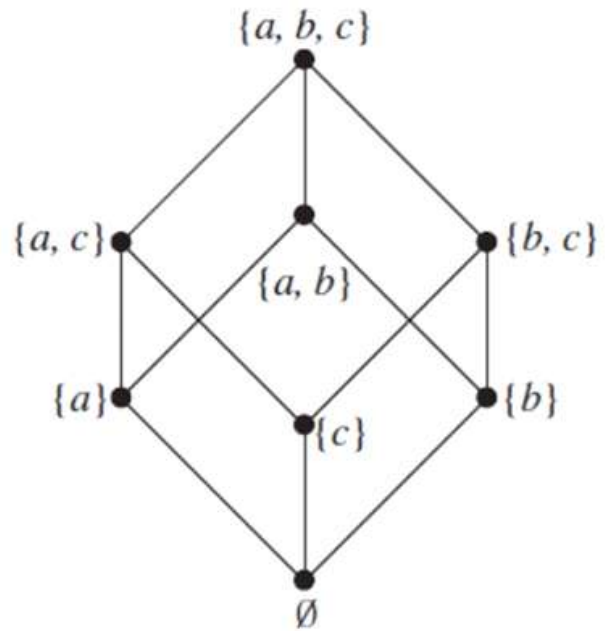
<예제9>

= 사전식 순서

<예제12>



<예제13>



= 하세도표를 3D로도 그릴 수 있습니다.

<예제14>

= 극대가 12 , 20 , 25 이고 , 극소가 2 , 5 입니다.

<예제15>

= 극대와 극소를 찾고 각각 유일무이하면 그게 최대 또는 최소가 됩니다.

<예제17>

= 최소는 당연히 1이고 , 최대는 극대가 너무 많으므로 존재하지 않음.

<예제18>

{a,b,c}의 모든 원소 보다 크거나 같은 원소는 e , f , j , h입니다.

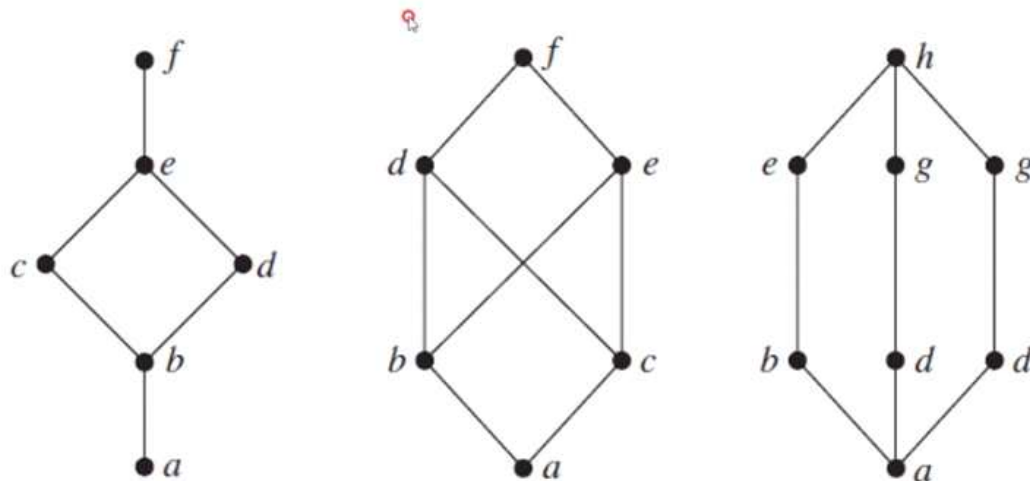
만약 : c = 100 , e = 101 , f = 102 h = 103 , g = 10 , d = 9 일수 있음.

{a,b,c}의 모든 원소 보다 작거나 같은 원소는 a 뿐입니다.

<예제19 , 20>

= 예제18을 잘 이해했다면 쉽게 풀 수 있습니다.

<예제21>



- 그림 1,3은 격자임
- 그림 2는 (b,c)를 잡을 때 최소 상한계가 d 와 e중에 구할 수 없으므로 격자 아님.

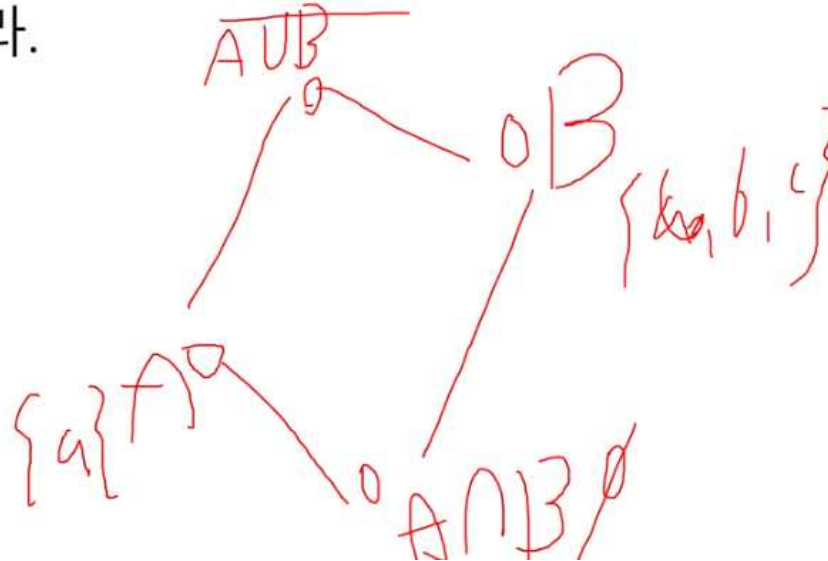
<예제 22>

= 격자임.

= 최대공약수와 최소공배수 개념

<예제24>

- S 가 집합일 때 $(P(S), \subseteq)$ 가 격자인지를 결정하라.



<10-0강_그래프 응용 사례> 10/28

*** 그래프 응용 분야이며 , 교양처럼 가볍게 수강했습니다. ***

1. Image Segmentation(이미지 분할)

= 이미지의 픽셀을 그래프로 활용합니다.

- 이미지는 엄청 많은 pixel로 이루어져 있습니다.

- 그래프의 Node : Pixel

- 그래프의 Edge : Pixel간의 연결 관계

- Cut : 그래프를 잘라서 두 개의 그래프로 나눕니다.

- 신기한 예시 영상들을 보여주셨습니다.

ex) 픽셀별로 밝기를 노드에 저장하고 , 밝기 기준으로 밝기가 밝은 부분만 사진에서 표현할 수 있습니다.

ex) 두 개의 객체 별로 이미지를 분할 할 수 있었습니다.

2. Motion Graph

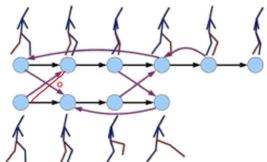
= 장면들이 모여서 만들어진 영상(Animation Sequence)을 그래프로 활용합니다.

- 그래프의 Node : 영상에 나오는 객체의 모션

- 그래프의 Edge : 영상에 나오는 객체 간의 관계

- 용량이 매우 큽니다.

<Example : 걷기>



- 영상에 나오는 사람을 Skeleton으로 하고 노드로 설정합니다.
- 노드 간에 관계를 edge로 표현했습니다.

- 신기한 예시 영상들을 보여주셨습니다.

3. Graph Matching

= 2개 이상의 이미지를 match 할 때 그래프를 이용합니다.

- 그래프의 Node : 이미지에서 특징이 중요한 pixel
- 그래프의 Edge : pixel간의 연결 관계
- 두 개의 이미지를 그래프를 사용하여 matching하는 사례를 보여주셨습니다.

<10-1강_그래프 응용 사례> 10/29 ~ 11/02

1. Graph

정의 1. 그래프 $G = (V, E)$ 는 공집합이 아닌 정점들 (노드들)의 집합 V 와 모서리들의 집합 E 로 구성한다. 각 모서리는 끝점이라 부르는 하나 또는 두 개의 정점들을 가진다. 모서리는 이 끝점들을 연결한다고 한다.

- Vertex = Node = 정점 = 노드
- Edge = Relation = 모서리 = 관계
- 단순그래프 = 노드와 노드 사이의 Edge가 최대 1개
- 멀티그래프(다중그래프) = 노드와 노드 사이의 Edge가 2개 이상
- 의사그래프 = 루프 + 다중 edge
- Directed Graph = 방향이 있는 그래프
- Undirected Graph = 방향이 없는 그래프

2. 문제 풀이

<예제 1>

- 세계인의 그래프에서 node는 60억개 , edge는 1조개.

<예제 2>

- 영향력 그래프는 방향성 그래프로 표현합니다.

<예제 3>

- 그래프의 예시

<예제 4>

- 통신에서 그래프를 많이 사용합니다.

<예제 5>

- 인터넷도 그래프 입니다.

<예제 6>

- 논문에서의 인용도 그래프 입니다.

<예제 7>

- 소프트웨어 설계는 그래프로 나타낼 수 있습니다.

<예제 8>

- 방향성 그래프는 의존도를 나타낼 수 있습니다.

<예제 9 ~ 예제 14>

- 예시 설명

<연습문제 3 ~ 9>

3. = 단순그래프

4. = 멀티그래프

5. = 의사그래프

6. = 멀티그래프

7. = 방향성 단순 그래프

8. = 방향성 다중 그래프

9. = 방향성 다중 그래프

<연습문제 12>

= Edge가 있으면 당연히 대칭을 만족하고 , 루프를 갖기 때문에 당연히 반사를 만족합니다.

<연습문제 14>

= Hawk와 연결된 Raccoon , Owl , Crow

<연습문제 18>

= Yvonne , Deborah는 Fred에게 영향을 줄 수 있고 , Fred는 Brian에게 영향을 줄 수 있습니다.

<연습문제 20>

= 4는 3을 이겼습니다.

= 1 , 2 , 5 , 6 은 4를 이겼습니다.

<연습문제 24>

= E-Mail의 특성입니다.

- 방향성 : A가 B에게 보낼 수 있고 , B도 A에게 보낼 수 있습니다.
- 다중 모서리 : A가 B에게 2통 이상의 메일을 보낼 수 있습니다.
- 루프 : A가 A에게 메일을 보낼 수 있습니다.

<연습문제 28>

= 지하철의 특성입니다.

- 방향성 , 비방향성이 애매합니다.
- 다중 모서리일 필요가 없습니다.
- 루프는 말이 되지 않습니다.

<연습문제 32>

= S6전에는 S3 , S1 , S4, S2가 실행되어야 합니다.

<10-2강_그래프 용어와 특별한 그래프 이론> 11/05 ~ 11/10

1. 그래프

1) Undirected Graph(비방향성 그래프)

정의 1. 비방향성 그래프 G 에서 두 정점 u 와 v 가 G 의 모서리의 끝점이라면 u 와 v 는 **인접한다**(또는 **이웃**)라고 한다. e 가 $\{u, v\}$ 와 관련되면 모서리 e 는 정점 u 와 v 에 **붙어있다**라고 한다. 모서리 e 는 u 와 v 를 **연결한다**고도 말한다. 정점 u 와 v 는 $\{u, v\}$ 와 연관된 모서리의 끝점들로 부른다.

정의 2. $G = (V, E)$ 의 한 정점 v 의 모든 이웃의 집합을 $N(v)$ 로 표기하고 v 의 **이웃**이라 한다. A 가 V 의 부분집합이라면 $N(A)$ 는 A 내의 한 정점에 인접한 모든 정점들의 집합이다.

$$N(A) = \bigcup_{v \in A} N(v)$$

2) Degree(차수)

정의 3. 비방향성 그래프에서 정점의 차수(degree)는 그것에 붙어 있는 모서리들의 수이다. 루프는 정점 차수에 2번 가산한다. 정점 v 의 차수는 $\deg(v)$ 로 표기한다.

차수가 0인 정점은 **고립되었다**라고 한다. 고립된 정점은 어떤 정점에도 인접하지 않는다. 정점의 차수가 1이면 그 정점은 **늘어져 있다**. 결론적으로 늘어진 정점은 정확하게 하나의 다른 정점과 인접한다.

정리 2. 비방향성 그래프에는 짝수개의 홀수 차수의 정점들이 있다.

3) Directed Graph(방향성 그래프)

정의 4. (u, v) 가 방향성 모서리를 갖는 그래프 G 의 모서리일 때 방향성 모서리 u 는 v 에 **인접한다**라고 하고 v 는 u 로부터 **인접된다**라고 한다. 정점 u 는 (u, v) 의 **시작 정점**이라 하고 v 는 (u, v) 의 **종료** 혹은 **끝 정점**이라 한다. 루프의 시작 정점과 끝 정점은 같다.

정의 5. 방향성 모서리를 갖는 그래프에서 정점 v 의 입력차수는 $\deg^-(v)$ 로 표기하고 종료 정점으로서 v 가 갖는 모서리의 수이다.

v 의 출력차수는 $\deg^+(v)$ 로 표기하고 시작 정점으로서 v 가 갖는 모서리의 수이다 (한 정점에서 루프는 그 정점의 입력차수와 출력차수 둘 다에 1을 기여한다.)

<입력차수>

= 정점으로 들어오는 화살표

<출력차수>

= 정점에서 나가는 화살표

2. 이분 그래프

1) 이분 그래프

이분 그래프

정의 6. 단순 그래프 G 에서 그 정점들의 집합 V 가 두 개의 겹치지 않는 집합 V_1 과 V_2 로 분리되고, 그래프의 모든 모서리는 V_1 의 한 정점과 V_2 의 한 정점을 연결하면, 이 단순 그래프는 **이분되었다**고 한다.

(그래프 G 에서 V_1 의 두 정점 간에 혹은 V_2 의 두 정점 사이를 연결하는 모서리는 존재하지 않는다)

이 조건이 만족되면 정점 쌍(V_1, V_2)을 G 의 정점집합 V 의 **이분**이라고 한다.

- 같은 그룹의 정점끼리는 edge가 존재하면 안 됩니다.
- 이분 되는 경우를 1개라도 찾으면 이분입니다.



A 노트북



B 노트북



C 노트북

○ 동빈 나는 A, B, C 노트북 다 괜찮아.

○ 태일 나는 A 노트북을 원해.

○ 한울 나는 B 노트북을 원해.

- 알고리즘 문제 풀 때 집합 X -> 집합 Y로

- 알고리즘 문제에서 생각해보면, 사람의 집합(동빈, 태일, 한울)과 노트북의 집합(A, B, C)을 연결 시켜 주는 이분 매칭 문제를 풀어본 적이 있습니다.

- 그 때 A, B, C, 동빈, 태일, 한울 모두 하나의 그래프로 생각해보면 노드입니다. 하지만 사람인 동빈, 태일, 한울은 사람의 집합으로, 노트북인 A, B, C는 그래프에서 두 개의 집합으로 이분할 수 있습니다.

<Bipartite Matching>

= 이분매칭

= 이분 그래프에서 두 개의 집합 끼리 매칭을 하는 방법

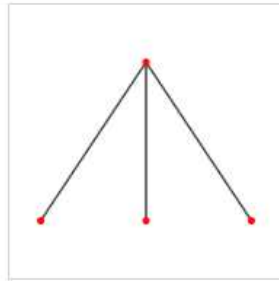
- 자세한 매칭 방법은 다루지 않겠습니다.

- <https://blog.naver.com/ndb796/221240613074>

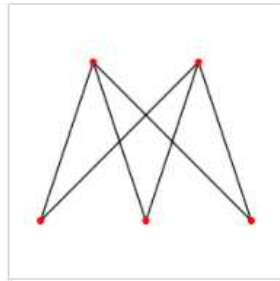
2) 완전 이분 그래프

= 이분 그래프를 만족하므로 집합 X , 집합 Y 로 나누어집니다. 집합 X 에 존재하는 모든 정점들이 집합 Y 에 존재하는 모든 정점들과 각각 모두 edge를 갖는 그래프입니다.

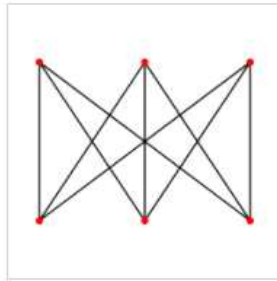
= $K(m,n)$



$K_{3,1}$



$K_{3,2}$



$K_{3,3}$

3) 이분 그래프의 필요충분조건

그래프가 이분 그래프일 필요충분조건은 한 정점에서 홀수개의 서로 다른 모서리들을 지나 이 정점으로 돌아오는 것은 불가능할 경우이다.

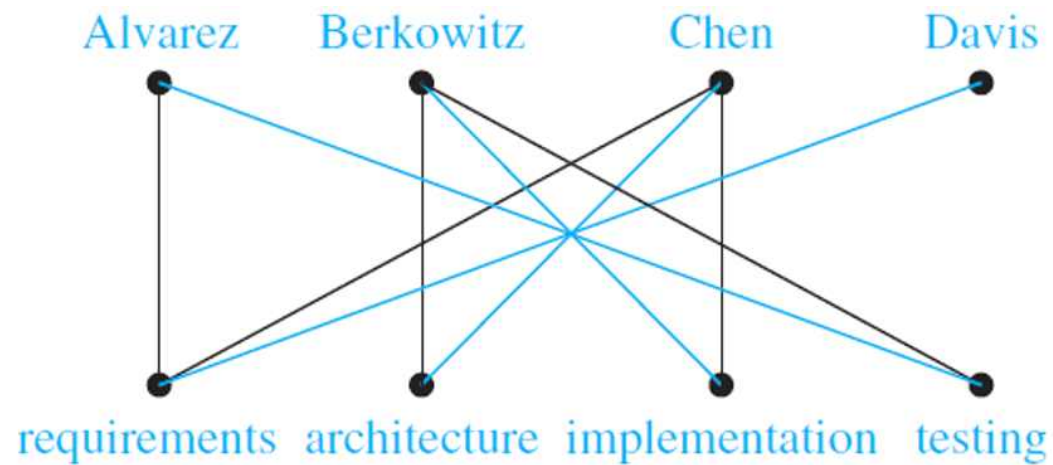
4) 최대 매칭과 완전 매칭

① 최대 매칭

= ?

② 완전 매칭

= 그래프의 모든 노드가 edge를 갖는 매칭



- Edge들을 잘 선택해서 모든 노드가 짝을 이룰 수 있게 만들면 완전 매칭입니다.

- 만약 Alvarez와 Berkowitz가 각각 1개의 Edge만 있고 둘 다 testing과 연결 되어 있다면 Alvarez가 testing을 선택하면 Berkowitz는 짝을 이루지 못하기 때문에 완전 매칭이 아닙니다.

- 파란색 Edge가 완전 매칭을 구성하는 Edge입니다.

3. Sub-Graph

= 부 그래프

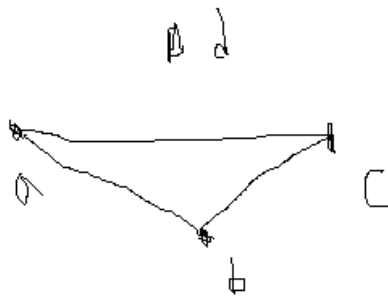
= 어떤 그래프의 일부분으로 만든 새로운 그래프

그래프 $G = (V, E)$ 의 부그래프는 $W \subseteq V$ 이고, $F \subseteq E$ 인 그래프 $H = (W, F)$ 이다. $H \neq G$ 이면 부그래프 H 는 G 의 진 부그래프이다.

$G = (V, E)$ 는 단순 그래프라 하자. 정점 집합 V 의 부분집합 W 에 의해 유도된 부그래프는 그래프 (W, F) 이다. 여기서 모서리 집합 F 는 모서리의 두 끝점이 W 에 속하는 E 의 모서리를 포함한다.

- V : 기존 그래프의 정점
- W : Sub-Graph의 정점
- E : 기존 그래프의 edge
- F : Sub-Graph의 edge
- 극단적인 경우 E 와 F 가 같을 수 있습니다.

<Example : E 와 F 가 같은 경우>



- $V : \{a, b, c, d\}$
- $W : \{a, b, c\}$
- Edge는 같습니다.

4. 모서리 제거 / 추가

- Edge는 추가 , 제거 , 축소할 수 있습니다.
- 정점을 제거 할 수 있습니다. 정점을 제거하면 관련된 edge도 사라집니다.

두 개의 단순 그래프 $G_1 = (V_1, E_1)$ 과 $G_2 = (V_2, E_2)$ 의 합집합은 정점들의 집합 $V_1 \cup V_2$, 모서리들의 집합 $E_1 \cup E_2$ 를 갖는 단순 그래프이다.
 G_1 과 G_2 의 합집합은 $G_1 \cup G_2$ 로 표기한다.

= 두 개의 그래프의 정점과 모서리를 합칩니다.

5. 문제 풀이

<예제 1>

- 루프를 2번 가산하는 것을 주의해서 차수 계산을 합시다.

<예제 2>

$$2m = \sum_{v \in V} \deg(v)$$

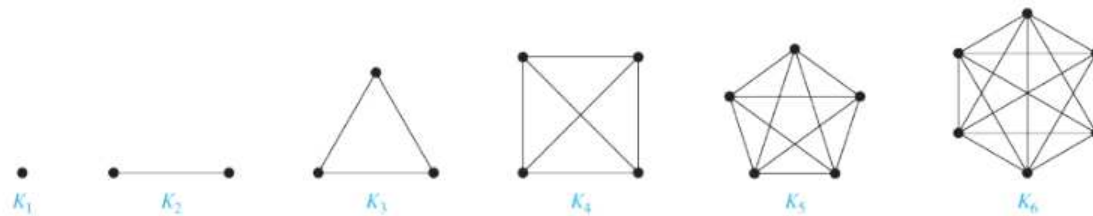
- 모든 차수의 합은 Edge의 개수 x 2 입니다. 그러므로 비방향성 그래프에서 차수의 합은 언제나 짝수 입니다.

<예제 3>

= 30개의 Edge가 존재합니다.

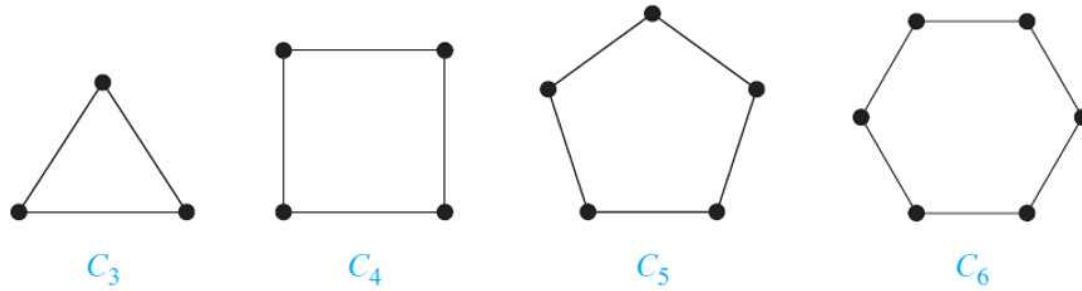
<예제 5>

= 완전 그래프 (K_n)



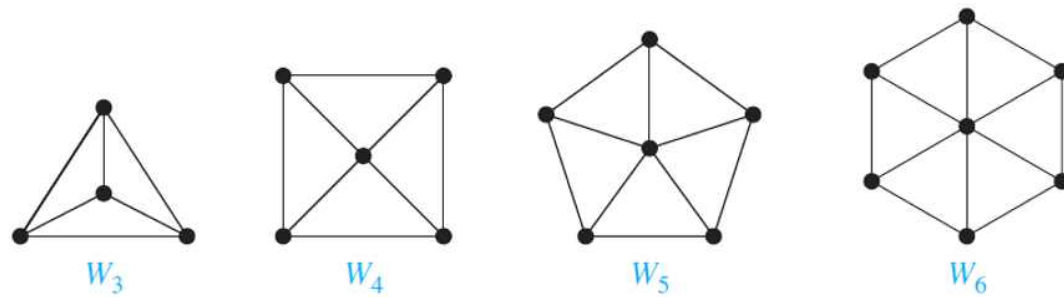
<예제 6>

= Cycle (C_n)



<예제 7>

= Wheel (W_n) : 사이클의 가운데에 점을 추가하고 다른 정점과 모두 연결합니다.



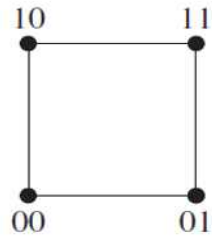
<예제 8>

= n-큐브 (Q_n) : 인접한 노드는 자신과 비트 1개만 다릅니다.

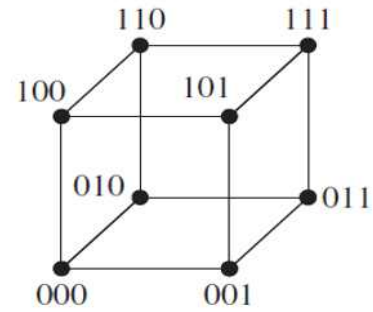
- n의 개수가 곧 모든 노드에서의 차수와 같게 됩니다.



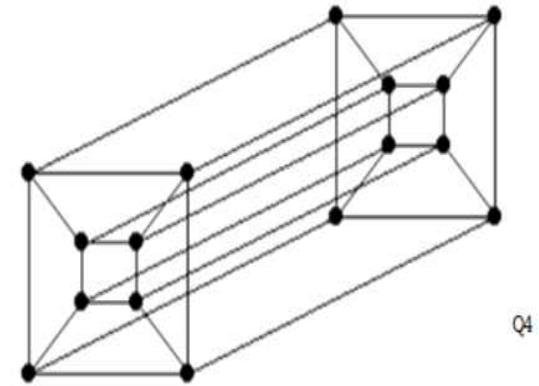
Q_1



Q_2



Q_3



Q_4

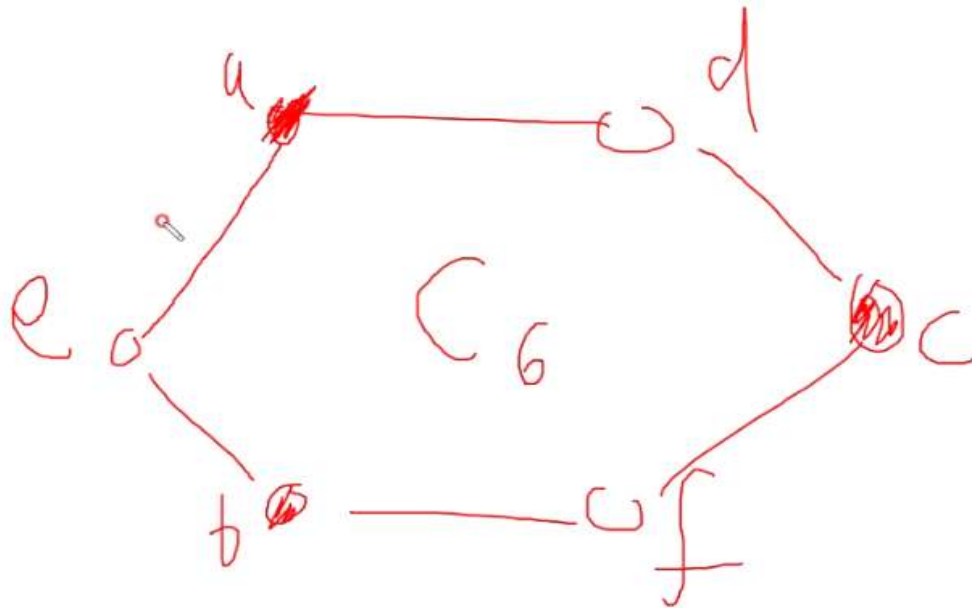
Q_1 = 길이 1의 비트

Q_2 = 길이 2의 비트

Q_3 = 길이 3의 비트

Q_4 = 길이 4의 비트

<예제 9>



- 집합 A : {a ,b ,c}
- 집합 B : {d ,e ,f}
- A와 B로 이분을 하면 서로 다른 집합 끼리는 edge가 존재
- A의 내부 원소끼리는 edge가 존재하지 않습니다.
- B의 내부 원소끼리는 edge가 존재하지 않습니다.

<예제 10>

= 어떤 방법으로 그래프를 나눠도 한 집합에서 edge가 발생합니다.

<예제 11>

1) G

= a를 기준으로 연결되어 있는 점을 고르면 {g,f,e,c} 입니다. 연결되어 있지 않은 점을 고르면 {a,b,d}입니다.
두 집합 내부에서 각각의 점들은 edge가 없기 때문에 이분그래프 입니다.

2) H

= f를 기준으로 연결되어 있는 점을 고르면 a,b,c,d,e 입니다. a-b가 있으므로 이분 그래프가 아닙니다.

<예제 13>

= 점이 m 개 n 개로 나누어진 집합일 때

<연습문제 1 ~ 3>

= 기초개념 문제이기 때문에 넘어갑시다.

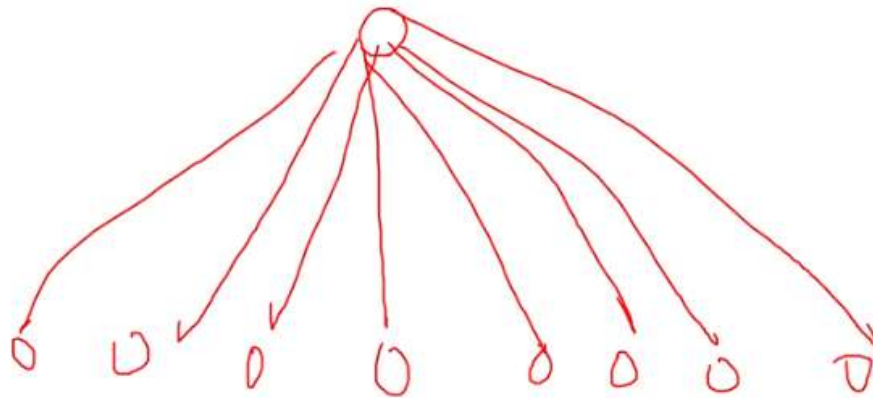
- 고립정점 = 따로 혼자 있는 정점

<연습문제 7 ~ 9>

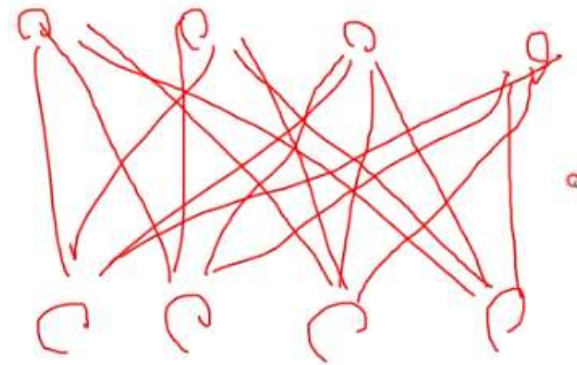
= 기초 차수 문제입니다.

<연습문제 20>

= 예제 5 ~ 8을 참고합시다. 그래프를 만드시 아름답게 그릴 필요는 없습니다.



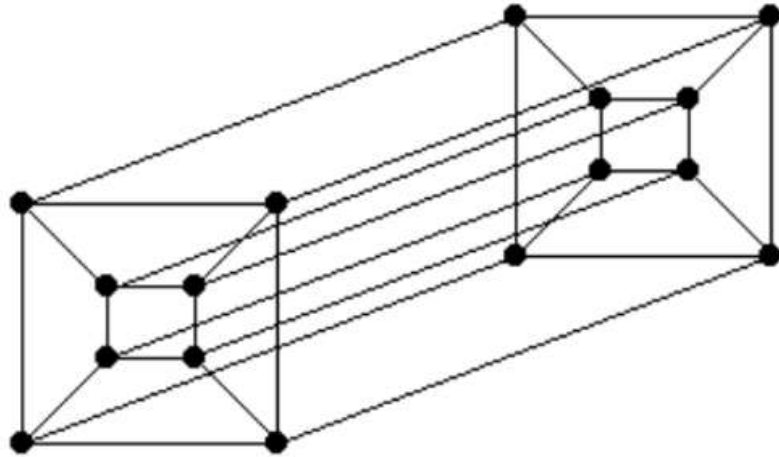
$K_{1,8}$



$K_{4,4}$

*** Q4 ***

= Q3 2개를 연결 합니다.



Q4

<연습문제 21 ~ 25>

- 21번 : $\{e\}$, $\{a,b,c,d\}$
- 22번 : $\{b,d,e\}$, $\{a,c\}$
- 23번 : 이분 그래프 X
- 24번 : $\{a,b,d,e\}$, $\{c,f\}$
- 25번 : 이분 그래프 X

<연습문제 26>

- 그래프 형태에 삼각형이 있으면 절대 이분 그래프가 될 수 없습니다.

a) K_n

= n 이 2일 때만 이분그래프 입니다.

b) C_n

= n 이 짝수 일 때만 이분그래프 입니다. 하지만 C_n 은 사이클이므로 n 이 3보다 커야하기 때문에 n 이 2일 때는 제외합니다.

c) W_n

= 어떠한 경우도 삼각형이 발생하므로 이분그래프가 될 수 없습니다.

d) Q_n

= 항상 이분그래프 입니다.

= Tip) 1의 개수가 홀수인 점들을 같은 집합으로 만들어서 검사해봅니다.

<연습문제 34>

= 완전 그래프의 부 그래프는 모두 완전 그래프 입니다.

= K_5 를 그려보고 일부 점만 선택하면 K_4 , K_3 , K_2 형태의 그래프가 탄생합니다.

<연습문제 36>

= 차수 순서 : 차수가 큰 노드부터 나열합니다.

ex) 21은 41111

<연습문제 38>

= 이분 그래프의 차수순서

1) $m \geq n$

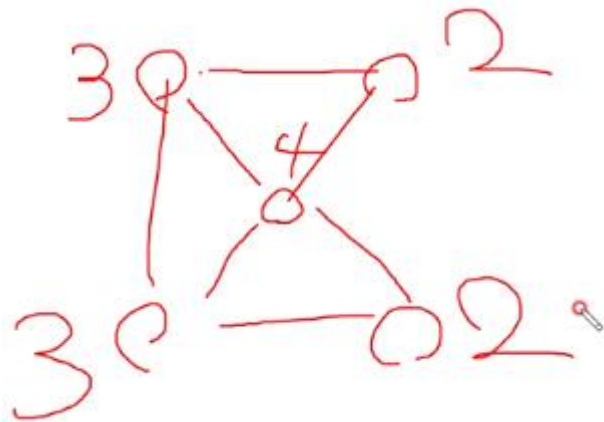
= mmmm nnnnnn

2) $n > m$

= nnnnn mmmmm

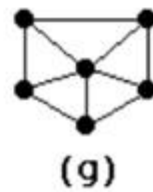
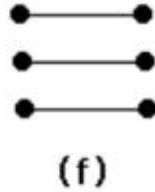
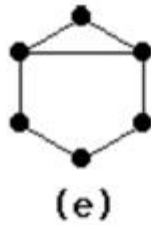
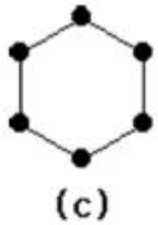
<연습문제 40>

- 모서리의 수 : 7



<연습문제 42>

- 홀수차수가 짝수 개 있어야 합니다.
- 노드가 6개인 그래프에서 차수가 5인 노드가 1개 있으면 차수가 0인 그래프는 존재할 수 없고, 차수가 5인 노드가 2개 있으면 차수가 1인 그래프는 존재할 수 없습니다. : a, h



<연습문제 48>

- 4가지

<연습문제 60>

- 그래프 G의 여집합 그래프 = G'
- $G \cup G' = K_n$
- 완전 그래프의 모서리 개수 : $n(n-1) / 2$
- G가 15개의 모서리를 갖는다면 $n * n-1 = 28 * 2 \rightarrow n$ 은 8 입니다.

<연습문제 64>

- 이분 단순 그래프라면 $edge \leq vertex^2 / 4$ 을 항상 만족합니다.

<10-3강_그래프 표현방법과 동형 그래프> 11/10 ~ 11/14

1. 그래프 표현 방법

1) 인접리스트

= 인접한 정점을 나열합니다.

2) 인접행렬

= 정점_i에서 정점_j로 가는 모서리가 있으면 1, 없으면 0으로 행렬을 만듭니다.

<인접리스트 vs 인접행렬>

- n : 노드의 개수
- m : 하나의 노드에서 최대 차수 (ex : 차수가 4, 3, 2, 2 면 m은 4입니다.)
- 보통 Edge의 개수와 m은 비례합니다. Edge가 많다면 노드에서의 차수들은 당연히 값이 커집니다.
- 인접리스트 = $m * n$ 보다 작거나 같은 개수의 메모리를 사용합니다.
- 인접행렬 = n^2 의 메모리를 사용합니다.

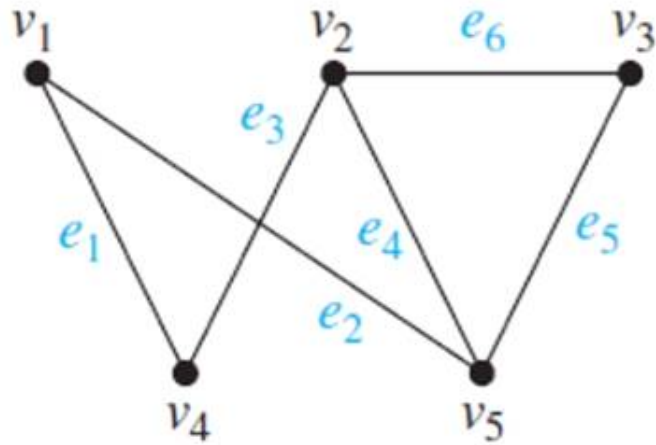
① Edge의 개수가 적은 경우

= m의 값이 작을 것이므로 상대적으로 인접리스트가 메모리를 적게 사용합니다.

② Edge의 개수가 많은 경우

= m의 값이 클 것이므로 상대적으로 인접행렬이 메모리를 적게 사용합니다.

3) 결합행렬



	e_1	e_2	e_3	e_4	e_5	e_6
v_1	1	1	0	0	0	0
v_2	0	0	1	1	0	1
v_3	0	0	0	0	1	1
v_4	1	0	1	0	0	0
v_5	0	1	0	1	1	0

2. 동형 그래프

단순 그래프들 $G_1 = (V_1, E_1)$ 과 $G_2 = (V_2, E_2)$ 는 V_1 에서 V_2 로의 함수 f 가 전단사 함수이고, V_1 의 모든 a, b 에 대해 a, b 가 G_1 에서 인접한다면 $f(a)$ 와 $f(b)$ 는 G_2 에서 인접하고 그 역도 성립하면 동형이다. 이때 함수 f 를 동형(isomorphism)이라 부른다.

<두 그래프가 동형일 조건>

- ① 같은 수의 정점
- ② 같은 수의 모서리
- ③ 각각의 정점들의 차수의 분포는 모두 같아야 합니다.
- ④ 10-4강에 나오는 단순 순환 OR 단순 경로의 길이(10-4강 연습문제 20), 단순 순환으로 찾는 것이 간편합니다.
ex) 그래프 G 의 차수 3이 2개, 2가 1개, 1이 3개면 H 의 차수도 차수 3이 2개, 2가 1개, 1이 3개여야 합니다.
 - 세 가지 중에서 한 가지도 만족하지 않으면 절대 동형이 되지 않습니다.
 - 세 가지를 모두 만족해도 동형이 아닌 경우가 존재합니다. -> 예제 10
 - 동형이면 인접행렬이 같을 수도 있고 다를 수도 있습니다.

3. 문제 풀이

<예제 1>

TABLE 1 An Adjacency List for a Simple Graph.	
<i>Vertex</i>	<i>Adjacent Vertices</i>
<i>a</i>	<i>b, c, e</i>
<i>b</i>	<i>a</i>
<i>c</i>	<i>a, d, e</i>
<i>d</i>	<i>c, e</i>
<i>e</i>	<i>a, c, d</i>

= 비방향성 그래프의 인접리스트 단점 : a-b는 a와 b에 모두 들어 갑니다. -> 중복으로 자료구조의 메모리 낭비

<예제 2>

= 예제1과 동일합니다.

<예제 3>

= 인접리스트와 인접행렬을 그려주셨습니다. 매우 간단합니다.

<예제 4>

= 인접행렬이 주어졌을 때 그래프를 그릴 수 있습니다.

<예제 5>

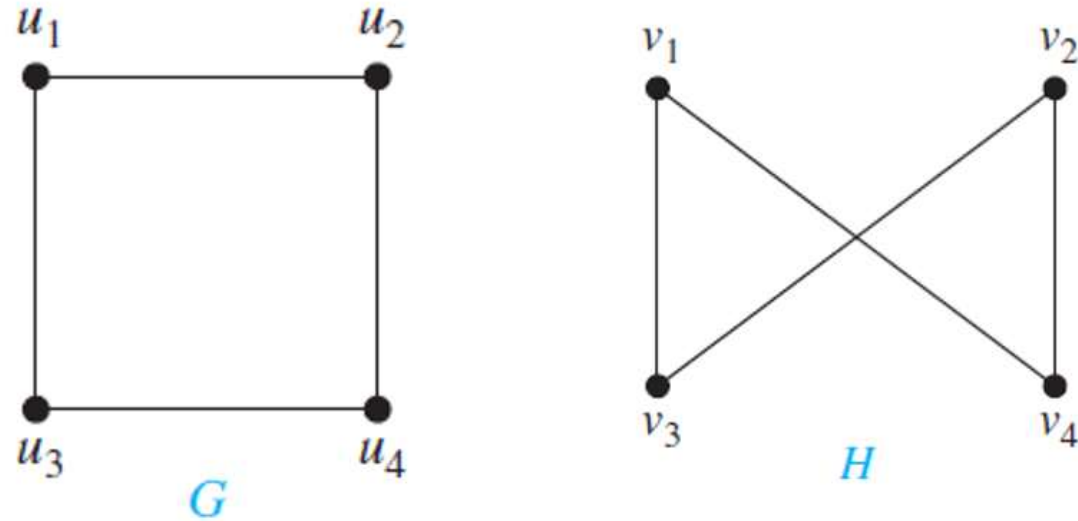
= 한 정점에서 모서리가 1개가 아닐 때에는 1에 해당 했던 값에 모서리 개수를 적자.

	a	b	c	d
a	0	3	0	2
b	3	0	1	1
c	0	1	1	2
d	2	1	2	0

<예제 7>

= 정점 마다 연결된 모서리를 행렬로 나타냅니다.

<예제 8>



- Index는 의미가 없습니다. 다시 말해 , u_1 과 v_1 을 대응시키고 , u_2 와 v_2 를 대응시키고 , u_3 와 v_3 를 대응시키고 , u_4 와 v_4 를 대응시킬 필요는 없습니다. (잘은 실수 발생!!!)
- 교수님께서서는 (u_1, v_1) , (u_2, v_4) , (u_3, v_3) , (u_4, v_2) 를 대응시키셨습니다. u_1 옆에 u_2 , u_3 이 있고 v_1 옆에 v_3 , v_4 가 있습니다.
- v_1 은 u_1 과 대응되고 , v_3 은 u_3 과 대응되고 , v_4 는 u_2 와 대응되는 노드이기 때문에 (u_1, v_1) 은 옳게 대응됩니다.
- 나머지 3개의 대응 관계도 직접 해보면 모두 알맞게 대응되기 때문에 G와 H는 동형 입니다.

<예제 9>

G의 최대 차수는 3인데 , H의 최대 차수는 4이므로 동형이 될 수 없습니다.

<예제 10>

- 정점의 개수 , 모서리의 개수 , 차수의 분포도 모두 같습니다.
- 하지만 H에서 차수가 3인 정점들끼리는 모두 어느 정도 인접해 있지만 G는 그렇지 않습니다.
- 그러므로 G와 H는 동형이 아닙니다.

<예제 11>

- 동형입니다.

<연습문제 2 , 4>

- 쉽습니다.

<연습문제 12>

- 비 방향성 그래프라면 반드시 대칭입니다.
- 해당 그래프는 대칭이 아니므로 방향성 그래프입니다.

<연습문제 14 , 16>

- Edge가 여러 개면 그 수만큼 행렬의 값으로 적어줍니다.

<연습문제 26>

- 결합행렬 나타내기.

<연습문제 32>

- n이 3 OR 4 일 때부터 구해보고 , n을 점점 증가시켜 봅니다.

1) K_n

= 대각선을 제외한 나머지가 모두 1.

$$\begin{bmatrix} 0 & 1 & 1 & \dots & 1 \\ 1 & 0 & 1 & \dots & 1 \\ 1 & 1 & 0 & \dots & 1 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & 1 & \dots & 0 \end{bmatrix}$$

2) C_n

= 한 노드에서 연결된 노드는 항상 2개 입니다. 또한 대각선을 기준으로 양 옆이 연결된 노드입니다.

ex) C_5 그래프에서 (0,0)이면 (0,1)이 1 (0,4)가 1.

$$\begin{bmatrix} 0 & 1 & 0 & \dots & 0 & 1 \\ 1 & 0 & 1 & \dots & 0 & 0 \\ 0 & 1 & 0 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & 0 & 1 \\ 1 & 0 & 0 & \dots & 1 & 0 \end{bmatrix}$$

3) W_n

= 설명 하시지 않았습니다.

$$\begin{bmatrix} 0 & 1 & 0 & \dots & 0 & 1 & 1 \\ 1 & 0 & 1 & \dots & 0 & 0 & 1 \\ 0 & 1 & 0 & \dots & 0 & 0 & 1 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & 0 & 1 & 1 \\ 1 & 0 & 0 & \dots & 1 & 0 & 1 \\ 1 & 1 & 1 & \dots & 1 & 1 & 0 \end{bmatrix}$$

4) $K(m,n)$

$$\begin{bmatrix} 0 & \dots & 0 & 1 & \dots & 1 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & 1 & \dots & 1 \\ 1 & \dots & 1 & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 1 & \dots & 1 & 0 & \dots & 0 \end{bmatrix}$$

= 완전 이분 그래프

- m 은 m 개의 원소로 이루어진 그룹_A
- n 은 n 개의 원소로 이루어진 그룹_B
- 그룹_A의 모든 원소는 그룹_B의 모든 원소와 연결됩니다.
- 그룹_B의 모든 원소는 그룹_A의 모든 원소와 연결됩니다.
- 같은 그룹끼리는 연결 되지 않습니다.
- 같은 그룹끼리는 0 , 다른 그룹끼리는 1

5) Q

= 공부하고 다시 듣기

<연습문제 34>

- 동형.

<연습문제 36>

- 왼쪽 그래프는 차수가 4 인 노드가 없습니다.

<연습문제 38>

- (u_3, v_2) , (u_4, v_3) , (u_2, v_5) , (u_1, v_1) , (u_5, v_5) 대응
- 동형

<연습문제 40>

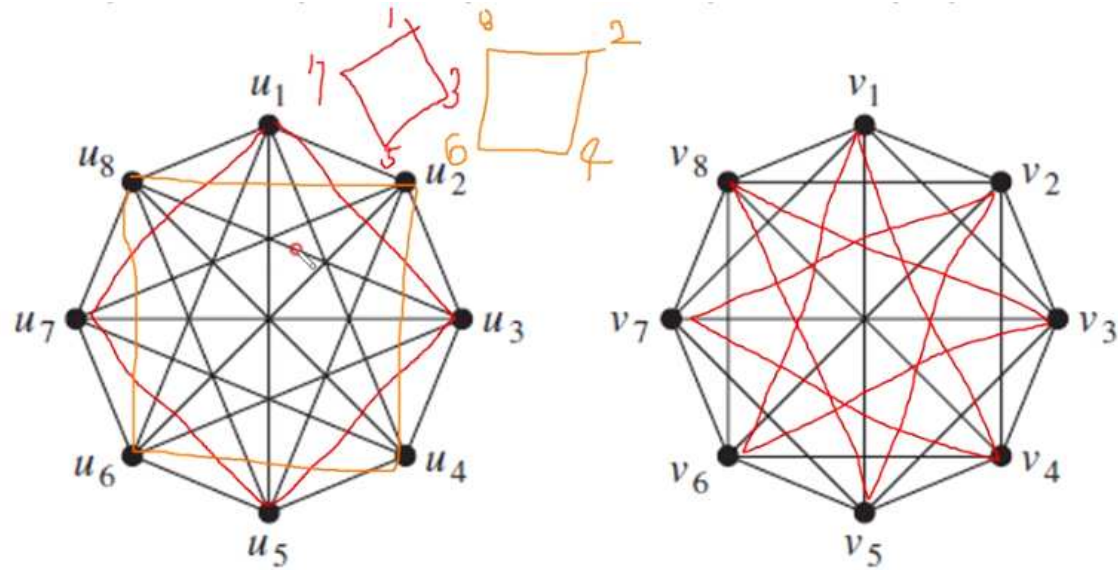
- 왼쪽 그래프는 차수가 4 인 노드가 없습니다.

<연습문제 42>

- 왼쪽 그래프는 차수가 4 - 4 - 3 형태
- 오른쪽 그래프는 차수가 4 - 3 - 4 형태
- 다시 말해 차수가 4인 노드끼리 인접한 그래프와 인접하지 않은 그래프
- 동형이 아닙니다.

<연습문제 44>

- 전체 연결($k=8$)에서 해당 그림을 각각 빼서 그려봅시다.



- 뺀 그림끼리 다르므로 동형이 아닙니다.
- 동형이라면 전체 연결에서 나를 빼도 동형이어야 합니다.

<연습문제 45>

- 동형은 동치관계입니다.

<연습문제 46>

- 고립 정점

<연습문제 49>

- 이분 그래프의 인접행렬은 당연히 완전 이분 그래프($K(m,n)$)과 비슷하지만 조금 다릅니다.
- 같은 그룹끼리는 반드시 모두 0이어야 하지만 다른 그룹끼리는 0이거나 1입니다,
- 완전 이분 그래프는 같은 그룹끼리는 반드시 0이고 다른 그룹끼리는 반드시 1입니다.

<연습문제 50>

- 자기-보수 : 여집합 그래프와 자신이 동형인 특수한 케이스

<연습문제 52>

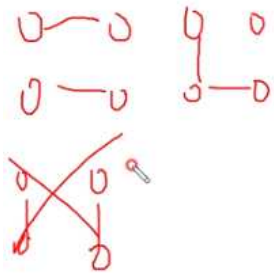
- 그래프가 G , 여집합이 $G-1$ 이면 G 합집합 $G-1 = U$ (전체집합)
- 이해 못하겠음.

<연습문제 54>

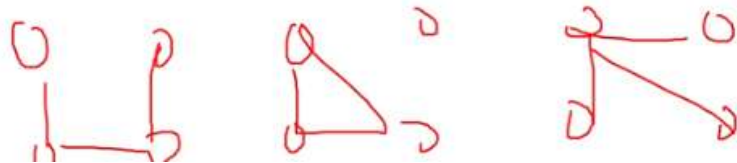
문제 / 모서리 개수	0개	1개	2개	3개	4개	5개	6개	총 개수
a) : 노드가 2개인 그래프	1	1	x	x	x	x	x	2
b) : 노드가 3개인 그래프	1	1	1	1	x	x	x	4
c) : 노드가 4개인 그래프	1	1	2	3	2	1	1	11

- 동형인 경우는 중복이므로 1개로 칩니다.

- c에서 모서리가 2개인 경우



- c에서 모서리가 3개인 경우



- c에서 모서리가 4개는 모서리가 2개인 경우의 여집합이므로 모서리가 2개일 때와 같습니다.

- c에서 모서리가 5개는 모서리가 1개인 경우의 여집합이므로 모서리가 1개일 때와 같습니다.

- c에서 모서리가 6개는 모서리가 0개인 경우의 여집합이므로 모서리가 0개일 때와 같습니다.

<연습문제 62 , 64>

- 방향성 그래프의 차수 비교는 deg-와 deg+를 각각 비교하여 동형인지를 파악합니다.

<연습문제 70>

인접리스트 = $2 * m$

인접행렬 = $n * n$

결합행렬 = $n * m$

<10-4강_연결성 이론> 11/14 ~ 11/23

1. 경로 , 순환 , 단순경로 , 단순순환

경로란 그래프의 한 정점에서 시작하여 항상 인접한 정점들의 쌍을 연결하며 그래프의 모서리들을 지나가는 일련의 모서리들임.

순환은 경로의 길이가 0보다 크고 시작 정점과 끝 정점이 같은 경로임.

경로나 순환은 정점들과 모서리들을 탐방한다.

단순 경로나 단순 순환은 같은 모서리를 한 번 이상 포함하지 않는 경로나 순환이다.

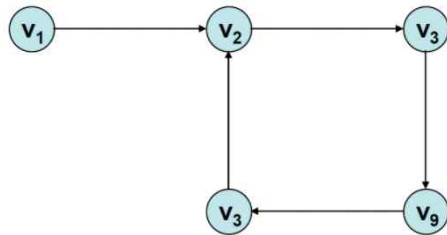
2. 연결(그래프)

connected

임의의 두 노드가 연결되었다(connected)는 것은 두 노드 사이에 경로가 존재한다는 이야기입니다. 이와 관련해 다음과 같이 정의됩니다.

- 모든 노드쌍 사이에 경로가 존재하는 무방향그래프는 연결되었다고 말한다.
- 임의의 방향그래프에서 방향을 무시하고 보면 연결되어 있을 경우, 해당 방향 그래프는 연결되었다고 말한다.
- 방향그래프의 임의의 노드쌍 a, b 에 대해 a 에서 b 로 가는 경로, b 에서 a 로 가는 경로가 존재한다면, 해당 방향그래프는 **강연결 (strongly connected)** 되었다고 말한다.

아래 방향그래프에서 방향을 무시하고 보면 이 그래프 내 임의의 노드쌍 사이에 모두 경로가 존재함을 알 수 있습니다. 따라서 아래 방향그래프는 *connected graph*입니다. 하지만 v_1 에서 v_3 로 가는 경로는 존재(v_1, v_2, v_3, v_9, v_3)하나, 반대로 v_3 에서 v_1 으로 가는 경로는 존재하지 않는다는 점에서 *strongly connected graph*는 아닙니다.



= 그래프에서 임의의 두 정점을 잡았을 때 어떠한 경우에도 두 정점 사이에 경로가 존재할 때 그 그래프를 **연결그래프**라고 부릅니다.

- 완전 그래프와는 잘 구분해야 합니다. 완전 그래프는 임의의 두 정점 사이에 Edge가 존재하는 그래프이고, 연결 그래프는 임의의 두 정점 사이에 경로가 존재하는 그래프입니다.

- **연결 그래프 = 약한 연결그래프**입니다.(비방향성 그래프에서의 개념) -> 11-4장에서 spanning tree를 공부할 때 나옵니다.

- 아래에서 배울 연결 요소는 연결그래프와 동등하게 생각해도 괜찮은 것 같습니다.(확실 X)

3. 연결 요소

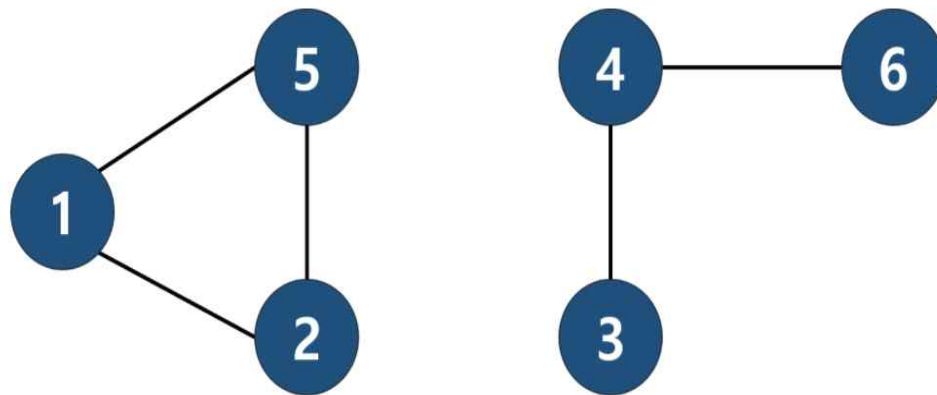
- 정점 1개가 혼자 떨어져 있어도 연결요소 입니다.

1) (약한) 연결 요소(Connected Component)

- **비방향성 그래프**에서의 연결 요소 개념입니다.
- 문제에서 방향성 그래프로 주어진다면 , 방향을 제거하여 비방향성 그래프로 생각하고 (약한) 연결 요소를 구합니다.

그래프 G의 연결 요소는 G의 다른 연결 부
그래프의 진 부그래프가 아닌 G의 연결 부
그래프이다.

즉, G의 연결요소는 G의 최대 연결 부그래
프이다.



- 6개의 노드로 이루어진 그래프를 다음과 같이 2개의 Sub-Graph로 나눌 수 있습니다.
- 결론부터 말하면 각각의 Sub-Graph({1,2,5}, {3,4,6})는 연결 요소 입니다.
- 하나의 연결요소에서 모든 node 끼리는 경로가 존재해야 합니다.
- 서로 다른 연결요소의 node 끼리는 edge가 존재하면 안 됩니다.
- 그래프에서 연결요소는 Disjoint-Set(=Union-Find)으로 찾을 수 있습니다.

2) 강한 연결 요소(Strongly Connected Component)

- 방향성 그래프에서의 연결 요소 개념입니다.
- 방향성 그래프에서 모든 정점이 모든 다른 정점에 대해 도달 가능한 경우입니다.

4. 그래프를 연결하는 방법

<절단 정점>

= 하나의 정점을 없앴을 때 그래프가 분할(연결 요소 증가)되는 정점

- 정점과 연결된 모든 edge를 삭제합니다.
- 언제나 모든 그래프가 절단 정점을 갖지는 않습니다.

ex) 완전 그래프(K_n)

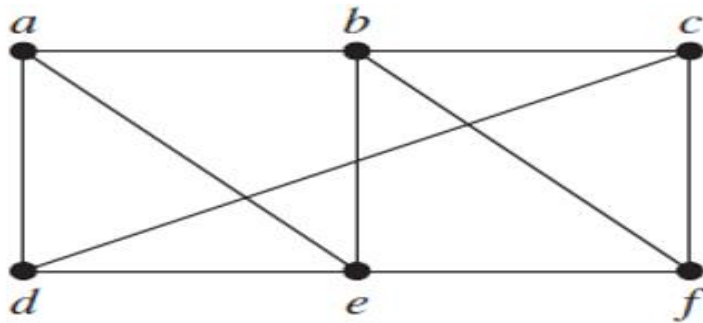
- 정점을 없애는 개념이기 때문에 혼자 독립적으로 존재하는 연결요소가 되지 않습니다.

ex) 정점 a를 없애면 정점 a가 혼자 따로 존재해서 연결요소가 되는 것이 아니라 그래프 세상에서 아예 사라지는 것입니다.

<절단 모서리>

= 하나의 모서리를 없앴을 때 그래프가 분할(연결 요소 증가)되는 모서리

<정점 절단>



- 절단 정점은 정점 1개를 없애는 것이라면, 정점 절단은 정점 여러 개를 없애는 것 입니다.
- 완전 그래프를 제외한 모든 그래프는 정점 절단이 존재합니다.
- 최소 개수의 정점들을 없애서 분할했을 때 정점의 개수를 카파(k)라고 합니다.
- b, c, e를 없애면 그래프가 분할됩니다.

5. 정점들 사이의 경로 수 세기

정리 2. G 를 정점이 $v_1, v_2, \dots, v_n$ 의 순서를 갖는 인접 행렬 A 를 갖는 그래프라 하자. (방향성 혹은 비방향성 모서리들, 다중 모서리들과 루프를 허용한다.) v_i 에서 v_j 로의 길이 r 인 다른 경로들의 수는 A^r 의 (i, j) 항과 같다.

- 예제 15에서 쉽게 확인할 수 있습니다.

6. 문제풀이

<예제 1>

- a, d, c, f, e : 경로이며 단순 경로(edge 중복 X)입니다.
- d, e, c, a : 경로가 아닙니다.
- b, c, f, e, b : 순환입니다.
- a, b, e, d, a, b : 단순 경로가 아닙니다.

<예제 4>

= 연결은 쉽습니다.

<예제 7>

- 절단 정점 : b , c , e
- 절단 모서리 : (a-b), (c-e)

<예제 8>

- G1 : c 제거
- G2 : c 제거
- G3 : {c,f} 제거
- G4 : {c,f} 제거
- G5 : {b,g,h} 제거
- 기재한 정점들의 집합을 제거하면 그래프가 분할됩니다.

<예제 9>

= 몇 개의 모서리를 삭제하면 그래프가 두 개로 나누어지는지 물어보는 문제입니다.

- G1 : 1개
- G2 : 2개
- G3 : 2개
- G4 : 3개
- G5 : 3개
- 최소 차수만큼의 모서리를 제거하면 두 개로 나누어집니다.

<예제 13>

- 동형을 찾을 때 단순 순환 경로의 길이를 추가해서 확인가능
- G는 단순 순환 경로의 길이가 4인 경로가 (u1,u6,u3,u2), (u6,u5,u4,u3) 뿐만 아니라 매우 많습니다.
- 그러나 H는 (v6,v5,v3,v2) 오직 1개 이므로 단순 순환 경로의 길이가 4인 경로의 개수가 두 그래프가 다르므로 동형이 아닙니다.

<예제 14>

- G의 u5를 u1,u2사이로 올리면 같다고 하셨습니다.

<예제 15>

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}, \quad A^4 = \begin{bmatrix} 8 & 0 & 0 & 8 \\ 0 & 8 & 8 & 0 \\ 0 & 8 & 8 & 0 \\ 8 & 0 & 0 & 8 \end{bmatrix}$$

- 우선 그래프 G에 대한 연결 관계를 행렬로 만들어 줍니다. 그 행렬을 A라고 합시다.
- A^4를 구합니다.
- a에서 d로 길이가 4인 경로를 행렬에서 찾으려면 8이므로 8개입니다.

<연습문제 6>

3) : 3개

4) : 1개

5) : 별모양 이여서 1개 같지만 2개.

<연습문제 12>

- 방향성 그래프에서 약한 연결을 확인하는 경우 , 방향성을 모두 제거합니다.
- a) : b에서 나가는 화살표가 없으므로 강한 연결은 아니지만 모든 정점이 연결되어 있으므로 약한 연결
- b) : 강한 연결이 되므로 , 약한 연결은 당연히 됩니다.
- c) : 강한 연결 X , 약한 연결 X

<연습문제 14>

- a) : (a,b,e), (c)
- b) : (c,d,e), (a), (b)
- c) : (a,b,c,d,f,g,h,i), (e)

<연습문제 20>

- 경로를 통해 아주 손쉽게 동형인지를 파악할 수 있습니다.
- 길이가 4인 경로가 G에서는 3개가 있지만 , H에서는 2개가 있으므로 동형이 아닙니다.

<연습문제 22>

- 많은 경로와 순환을 구했지만 모두 동일하므로 동형입니다.

<연습문제 26>

- 인접행렬을 만들고 A^2 , A^3 , A^4 , etc를 구해서 해결합니다.

<연습문제 30>

- 단순 그래프에서 홀수 차수의 정점은 짝수 개 입니다.

<연습문제 32>

- a를 없애도 그래프는 분할되지 않습니다. 정점을 없애는 개념이므로 a를 없애는 것은 a가 독립적인 연결 요소가 되는 것이 아닙니다.
 - 하지만 c나 d를 없애면 그래프가 분할됩니다.
- ex) c를 없애면 (a,b)와 (d,e,f)가 연결되지 않고 분할되어 연결요소가 증가합니다.

<연습문제 34>

- 31 : X
- 32 : (c-d)
- 33 : (a-b), (b-c), (c-d), (c-e), (e-i), (i-h)
- 절단 정점이나 절단 모서리는 반드시 각각 정점 1개 , 모서리 1개씩만 제거하는 개념입니다.

<연습문제 42>

- 단순 그래프 G1과 단순 그래프 G2의 합집합인 연결 단순 그래프 G가 존재한다면 , G1과 G2는 반드시 적어도 하나의 공통 정점을 갖습니다.

<연습문제 46>

- 연결 요소들을 3개 가진 그래프의 인접행렬을 만드는 경우 , 연결요소_1부터 인접행렬에 추가하고 연결요소_2 , 연결요소_3을 추가하면 연결요소_1의 구역 , 연결요소_2의 구역 , 연결요소_3의 구역으로 나뉘고 , 나머지는 모두 0이 됩니다.

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

<연습문제 48>

- 어떤 점을 드러내더라도 분할되지 않습니다.
- $C_n(n \geq 3)$ 은 절단 정점을 갖지 않습니다.
- $W_n(n \geq 3)$ 은 절단 정점을 갖지 않습니다.
- $K_{m,n}(m \geq 2, n \geq 2)$ 은 절단 정점을 갖지 않습니다.

<연습문제 50>

- 카파는 최소로 정점을 제거해서 그래프를 분할시키는 정점의 개수
- 람다는 최소로 모서리를 제거해서 그래프를 분할시키는 모서리의 개수

① a)

- 카파 : 1개
- 람다 : 2개

② b)

- 카파 : 1개
- 람다 : 3개

③ c)

- 카파 : 2개
- 람다 : 2개

④ d)

- 카파 : 4개
- 람다 : 4개

<연습문제 54>

- 즐거운 두뇌 학습

<연습문제 56>

- 인접행렬을 A라 합니다. $A^1 \rightarrow A^2 \rightarrow A^3 \rightarrow \text{etc}$ 를 해보면서 인접행렬에서 구하고자 하는 (i, j)가 최초로 0이 아닌 것을 경우를 찾습니다.
- 연습문제 58로 이해합시다.

<연습문제 58>

- a $\rightarrow$ c로의 최단경로의 길이를 육안으로 보면 3인 것을 쉽게 알 수 있습니다.
- 해당 그래프에 대해서 인접행렬을 그려보면 a의 index가 1이고, c의 index가 3이므로 1행 3열의 원소가 매우 중요합니다.
- A의 1행 3열의 원소는 0이고, 이는 a $\rightarrow$ c로 거리가 1인 경로가 없다는 것을 의미합니다.
- A^2 의 1행 3열의 원소는 0이고, 이는 a $\rightarrow$ c로 거리가 2인 경로가 없다는 것을 의미합니다.
- A^3 의 1행 3열의 원소는 1이고(최초로 0이 아님), 이는 a $\rightarrow$ c로 거리가 3인 경로가 존재함을 의미하고, 이는 a $\rightarrow$ c로의 최단경로가 3임을 나타냅니다.

<연습문제 62>

- G1은 모든 정점이 연결되어 있고 , G2는 일부 정점만이 연결되어 있습니다.
- 정점이 N개 일 때 정점 끼리 연결 되어 있다면 최대 거리는 N-1 입니다.
- 정점이 N개 일 때 1부터 N-1까지의 인접행렬을 모두 더한 행렬에서 자기 자신으로 돌아오는 부분(대각선)을 제외하고 0인 원소는 해당 정점끼리 어떤 경우에도 경로가 존재하지 않음을 의미합니다.

1) G1

$$A^3 = \begin{bmatrix} 2 & 3 & 5 & 2 & 1 & 2 & 1 \\ 3 & 2 & 5 & 2 & 1 & 2 & 1 \\ 5 & 5 & 4 & 6 & 1 & 6 & 1 \\ 2 & 2 & 6 & 2 & 3 & 5 & 1 \\ 1 & 1 & 1 & 3 & 0 & 1 & 1 \\ 2 & 2 & 6 & 5 & 1 & 2 & 3 \\ 1 & 1 & 1 & 1 & 1 & 3 & 0 \end{bmatrix}.$$

- 인접행렬을 A라고 하겠습니다.
- G1은 정점이 7개이므로 $A^1 \rightarrow A^2 \rightarrow A^3 \rightarrow A^4 \rightarrow A^5 \rightarrow A^6$ 까지 계산을 해봐야 합니다.
- 하지만 A^3 에서 자기 자신으로 돌아오는 대각선을 제외하고 모두 0이 아니므로(0이 아니면 연결되었다는 뜻)

2) G2

If we compute the sum $A + A^2 + A^3 + A^4 + A^5$ we obtain

$$\begin{bmatrix} 6 & 7 & 7 & 0 & 0 & 0 \\ 7 & 3 & 3 & 0 & 0 & 0 \\ 7 & 3 & 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 20 & 21 & 21 \\ 0 & 0 & 0 & 21 & 20 & 21 \\ 0 & 0 & 0 & 21 & 21 & 20 \end{bmatrix}$$

- 인접행렬을 A라고 하겠습니다.
- G2는 정점이 6개이므로 $A^1 \rightarrow A^2 \rightarrow A^3 \rightarrow A^4 \rightarrow A^5$ 까지 계산을 해봐야 합니다.
- 그림에서 $A^1 \sim A^5$ 까지 모두 더한 행렬을 확인할 수 있습니다.
- 행렬의 (1,4)가 0인 것을 보고 우리는 a와 d는 연결되지 않음을 알 수 있습니다.

<10-5강_오일러와 해밀턴> 11/18 ~ 11/22

- Path(경로) vs Circuit(순환, 회로)
- 4가지 이론(오일러 순환, 오일러 경로, 해밀턴 순환, 해밀턴 경로)을 완벽하게 구분해서 이해해야 합니다.
- 4가지 이론 모두 그래프의 하나의 시작 정점에서라도 이론을 만족하면 이론을 성립합니다.

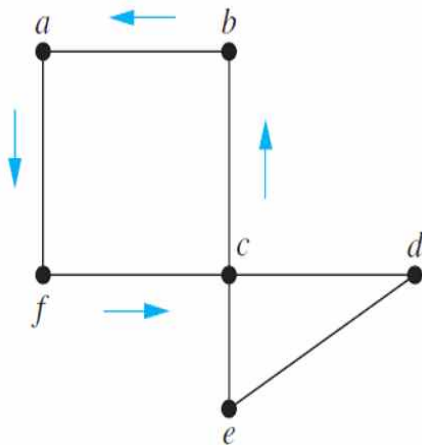
ex) 그래프 G가 6개의 정점을 갖고 있고 오일러 순환을 만족한다고 합시다. 그러면 G에서 단 하나의 시작 정점이라도 오일러 순환을 만족시키면 그 그래프 G는 오일러 순환이 성립하게 됩니다.

1. 오일러 순환(Euler Circuit)

= 어떤 그래프의 모든 모서리를 포함하는 단순 순환

= 그래프의 한 정점에서 시작하여, 그래프의 모든 모서리들을 정확히 한 번만 지나서 그 정점으로 돌아옵니다.

- 모든 정점의 차수가 짝수면 오일러 순환을 만족하고, 오일러 순환을 만족하면 모든 정점의 차수가 짝수인 필요충분조건입니다.
- 한붓그리기



- 들어오는 모서리가 있으면 나가는 모서리도 있어야 합니다.

ex) 쾨니스베르크의 7개 다리

2. 오일러 경로(Euler Path)

- = 어떤 그래프의 모든 모서리를 포함하는 단순 경로
- = 그래프의 한 정점에서 시작하여 , 그래프의 모든 모서리들을 정확히 한 번만 지나서 어떤 정점에 도착합니다.
- 홀수 차수의 정점이 정확히 2개만 있고 나머지 정점의 차수는 모두 짝수여야 합니다.

3. 해밀턴 순환(Hamilton Circuit)

- = 어떤 그래프의 모든 정점 포함하는 단순 순환
- = 그래프의 한 정점에서 시작하여 , 그래프의 모든 정점들을 정확히 한 번만 지나서 그 정점으로 돌아옵니다.
- 해밀턴 순환의 조건은 없습니다. 직접 찾아야 합니다.

ex) Traveling Salesman Problem(외판원 문제)

4. 해밀턴 경로(Hamilton Path)

- = 어떤 그래프의 모든 정점을 포함하는 단순 경로
- = 그래프의 한 정점에서 시작하여 , 그래프의 모든 정점들을 정확히 한 번만 지나서 어떤 정점에 도착합니다.

5. Summary

- 오일러는 모든 모서리를 한번 씩 지납니다.
- 오일러는 정점을 중복해서 지나도 무방합니다.
- 해밀턴은 모든 정점을 한번 씩 지납니다.
- 해밀턴은 모서리를 중복해서 지나도 무방합니다.

6. 문제

<예제 1>

- G1 : 오일러 순환 $O(a,e,c,d,e,b,a)$
- G2 : 오일러 순환 X , 오일러 경로 X
- G3 : 오일러 순환 X , 오일러 경로 $O(a,c,d,e,b,d,a,b)$

<예제 2>

- 방향을 반드시 고려해야 합니다.
- H1 : 오일러 순환 X , 오일러 경로 X
- H2 : 오일러 순환 $O(a,g,c,b,g,e,d,f,a)$
- H3 : 오일러 순환 X , 오일러 경로 $O(c,a,b,c,d,b)$

<예제 3>

- 한붓그리기가 가능합니다.

<예제 4>

- G1 : 2개의 정점에서만 차수가 홀수 입니다. 오일러 경로 $O(d,a,b,c,d,b)$
- G2 : 2개의 정점에서만 차수가 홀수 입니다. 오일러 경로 $O(b,a,g,f,e,d,c,b,g,c,f,d)$
- G3 : 6개의 정점에서 차수가 홀수 입니다. 오일러 경로 X

<예제 5>

- G1 : 해밀턴 순환 $O(a,b,c,d,e,a)$
- G2 : 해밀턴 순환 X , 해밀턴 경로 $O(a,b,c,d)$
- G3 : 해밀턴 순환 X , 해밀턴 경로 X

<예제 6>

- G : 정점 e를 지날 때 무조건 정점 d를 중복해서 지납니다.
- H : 정점 c를 무조건 여러 번 지납니다.
- 그러므로 G , H 모두 해밀턴 순환을 갖지 않습니다.

<예제 7>

- K_n 에서 정점들은 서로 모두 연결되어서 해밀턴 순환을 갖습니다.

<예제 8>

- 그레이 코드 : 화살표의 위치로 전송 신호가 무엇인지 알 수 있습니다.
- 인접한 호 사이에 하나의 비트만 다르도록 만든 코드입니다.

<연습문제 2>

- 오일러 순환 O
- 오일러 경로 X

<연습문제 4>

- 오일러 순환 X
- 오일러 경로 O

<연습문제 10>

- 그림을 보고 그래프 구조를 만듭니다.
- 오일러 순환 있습니까? 그렇습니다.

<연습문제 14>

- 한붓그리기(시작점과 끝점은 다름) : 오일러 경로
- 오일러 경로 0

<연습문제 16>

- 고립 정점을 갖지 않는 방향성 멀티그래프에서 약한 연결 그래프이고 입력차수와 출력차수가 같을 때 오일러 순환을 갖습니다.

<연습문제 17>

- 고립 정점을 갖지 않는 방향성 멀티그래프에서 약한 연결 그래프이고 입력차수와 출력차수가 다음 두 경우를 제외하고 모두 같은데 , 하나는 입력차수가 출력차수보다 1개 크고 , 다른 하나는 출력차수가 입력차수보다 1개 많으면 오일러 순환은 없고 오일러 경로는 갖습니다.

<연습문제 18 , 20 , 22>

- 연습문제 17에 관한 예제입니다.

<연습문제 26>

- n 에 따른 오일러 순환 성립 조건
- K_n : n 이 홀수
- C_n : $n \geq 3$
- W_n : 오일러 순환 X
- Q_n : n 이 짝수

<10-6강_최단경로> 11/25 ~ 11/29

1. Dijkstra Algorithm_독학

- 임의의 시작 노드에서 모든 노드까지의 최단경로
- 간선의 가중치가 음수인 경우는 해결 할 수 없습니다.
- 궁극의 목표 : 그래프의 모든 노드를 S에 채워 넣기입니다. 다시 말해 , 모든 노드의 최단 경로를 확정 시켜야 합니다.

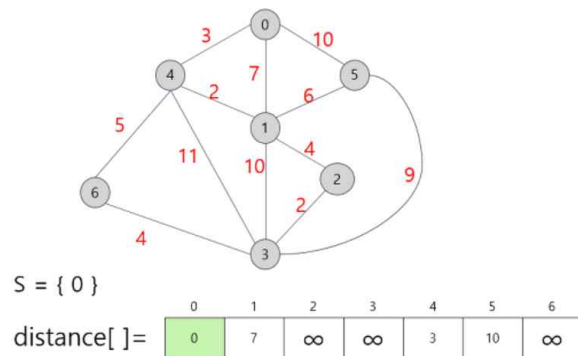
<자료구조>

- S[] : set(확정)입니다. 여기 저장되는 노드는 최종 최단경로를 확정 지었다는 의미입니다.
- distance[] : 시작 노드에서 해당 index(=node번호)까지의 현재까지의 최단경로
- 인접행렬 OR 인접리스트

*** 설명에서는 인접행렬을 사용했다고 가정하겠습니다. ***

*** 임의의 시작 노드를 Node_0 라고 가정하겠습니다. ***

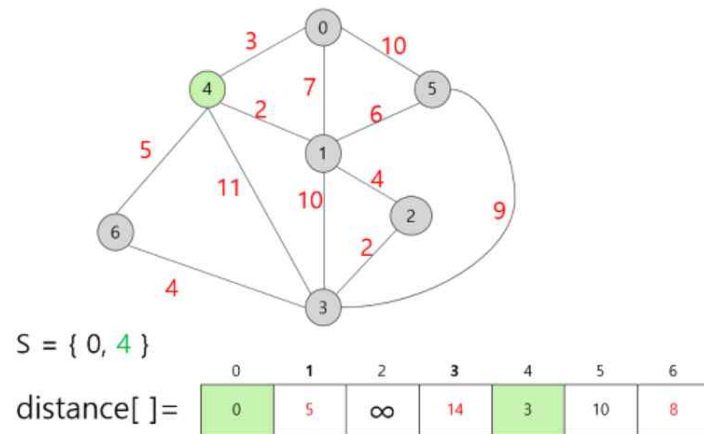
1) 1단계



- 시작한 정점은 default로 S에 추가시킵니다.
- 현재 알고 있는 정보는 인접행렬에서 주어진 인접한 노드와의 거리 밖에 없습니다.
- 이를 전부 distance에 갱신시킵니다.

2) 2단계

- 이 과정은 예시를 통해 설명하겠습니다.



- 모든 과정을 완벽하게 이해합시다.
- 이전에 갱신한 distance[]에서 S에 저장한 노드를 제외한 노드 중에 가장 작은 값을 갖는 노드를 고릅니다. S에 노드_0이 저장되어 있으므로 0을 제외한 노드_4가 distance[]에서 가장 작은 값을 갖기 때문에 노드_4를 고릅니다.
- 노드_4에서 인접한 노드들과의 거리(add_dis)를 각각 계산합니다.
- 기존의 distance[]에는 노드_4까지의 거리(3)가 저장되어 있고, 3과 add_dis를 더해서 새로운 거리(new_dis)를 각각 계산합니다.
- new_dis들과 기존의 distance[]를 비교해서 new_dis가 작으면 갱신합니다.

<그림 설명>

- 노드_4는 노드_0,1,3,6과 인접해 있습니다.

노드	기존 dis[]	3 + add_dis = new_dis
0	0	3 + 3 = 6
1	7	3 + 2 = 5
3	INF	3 + 11 = 14
6	INF	3 + 5 = 8

- 기존 dis[]보다 작은 new_dis는 노드_1,3,6 이므로 그들을 갱신시킵니다.

3) 3단계

- S에 모든 노드를 저장할 때 까지 2단계를 반복합니다.
- 갱신한 distance[]에서 S에 저장한 노드를 제외한 노드 중에 가장 작은 노드(Cur_Node)가 갖는 값은 시작노드인 노드_0에서 Cur_Node까지의 현재까지의 최단경로지만, 이 값이 결국 시작노드인 노드_0에서 Cur_Node까지의 최종 최단경로가 됩니다.

2. Dijkstra Algorithm_교수님 강의

ALGORITHM 1 Dijkstra's Algorithm.

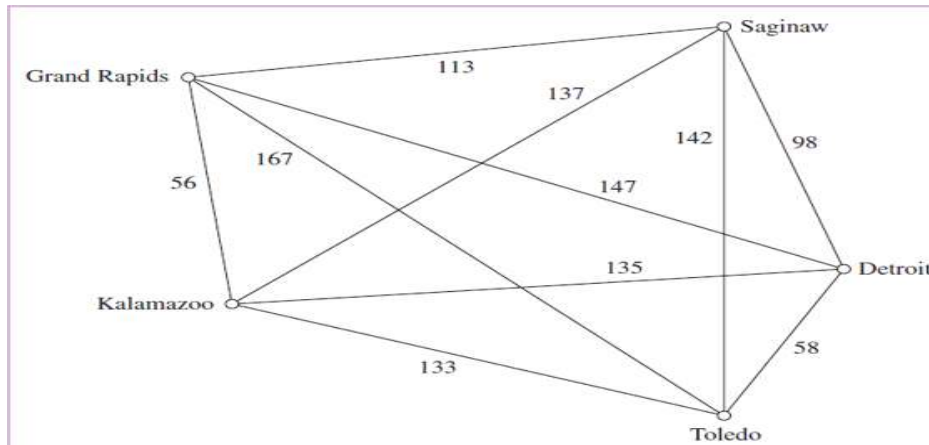
```
procedure Dijkstra( $G$ : weighted connected simple graph, with  
    all weights positive)  
{ $G$  has vertices  $a = v_0, v_1, \dots, v_n = z$  and lengths  $w(v_i, v_j)$   
    where  $w(v_i, v_j) = \infty$  if  $\{v_i, v_j\}$  is not an edge in  $G$ }  
for  $i := 1$  to  $n$   
     $L(v_i) := \infty$   
 $L(a) := 0$   
 $S := \emptyset$   
{the labels are now initialized so that the label of  $a$  is 0 and all  
    other labels are  $\infty$ , and  $S$  is the empty set}  
while  $z \notin S$   
     $u :=$  a vertex not in  $S$  with  $L(u)$  minimal  
     $S := S \cup \{u\}$   
    for all vertices  $v$  not in  $S$   
        if  $L(u) + w(u, v) < L(v)$  then  $L(v) := L(u) + w(u, v)$   
        {this adds a vertex to  $S$  with minimal label and updates the  
        labels of vertices not in  $S$ }  
return  $L(z)$  { $L(z)$  = length of a shortest path from  $a$  to  $z$ }
```

- 복잡도는 이중 루프로 인해 $O(n^2)$ 가 됩니다.

3. 외판원 문제

- n개의 도시를 정확히 한 번씩 방문하고 출발점으로 되돌아오는 문제
- 해밀턴 순환과 비슷한 내용입니다.

<Example : Detroit에서 출발하여 나머지 4개의 도시를 모두 한 번 씩 지나서 다시 Detroit로 돌아오는 경우의 수를 구해봅시다.>



Route	Total Distance (miles)
Detroit–Toledo–Grand Rapids–Saginaw–Kalamazoo–Detroit	610
Detroit–Toledo–Grand Rapids–Kalamazoo–Saginaw–Detroit	516
Detroit–Toledo–Kalamazoo–Saginaw–Grand Rapids–Detroit	588
Detroit–Toledo–Kalamazoo–Grand Rapids–Saginaw–Detroit	458
Detroit–Toledo–Saginaw–Kalamazoo–Grand Rapids–Detroit	540
Detroit–Toledo–Saginaw–Grand Rapids–Kalamazoo–Detroit	504
Detroit–Saginaw–Toledo–Grand Rapids–Kalamazoo–Detroit	598
Detroit–Saginaw–Toledo–Kalamazoo–Grand Rapids–Detroit	576
Detroit–Saginaw–Kalamazoo–Toledo–Grand Rapids–Detroit	682
Detroit–Saginaw–Grand Rapids–Toledo–Kalamazoo–Detroit	646
Detroit–Grand Rapids–Saginaw–Toledo–Kalamazoo–Detroit	670
Detroit–Grand Rapids–Toledo–Saginaw–Kalamazoo–Detroit	728

- 순열과 조합을 생각하면 $(4 * 3 * 2) / 2 = 12$ 가지의 경로가 나옵니다.

4. 문제풀이

<예제 1 , 2>

= 완벽한 Dijkstra 해법으로 하지 않으시고 대충 휴리스틱으로 해결하셨습니다.

<연습문제 14>

- 가중치가 없으면 점 사이에 경로하는 edge의 수

<연습문제 16>

- dijkstra는 최단 경로의 길이만 알 수 있지만 최단 경로를 알고 싶다면 dijkstra 탐색 과정 중에 ...

- 너무 대충 설명하셨습니다.

<연습문제 18>

- 모서리들의 가중치가 서로 다르다고 해도 가중치 그래프에서 두 정점 사이의 최단 경로는 유일하지 않습니다.

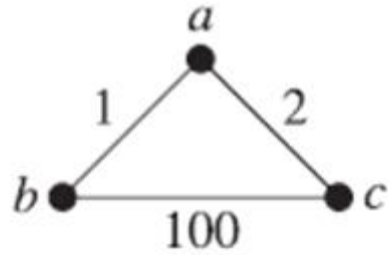
<연습문제 24>

- 음의 가중치를 가지면 dijkstra로 해결할 수 없습니다.

<연습문제 26>

- 점 5개이므로 $4!/2$ 가 나옵니다.

<연습문제 29>



<모든 정점을 한 번씩 방문하고 자기 자신으로 돌아오는 경로의 길이 중 최소>

① $a \rightarrow b \rightarrow c \rightarrow a = 1 + 100 + 2 = 103$

② $a \rightarrow b \rightarrow a \rightarrow c \rightarrow a = 1 + 2 = 3$

- 2번 방법이 자신을 자주 방문하지만 경로는 더 짧습니다.

<11-1강_트리 소개> 12/04

1. 트리의 정의

- = Cycle을 갖지 않는 비방향성 그래프.
- 트리도 결국 특수한 그래프입니다.
- 비방향성 그래프에서 정점들 간에 유일한 단순 경로가 존재하면 트리입니다.

<내부 정점 (Internal Vertex)>

- = 트리에서 자식을 갖는 모든 정점
- 루트도 내부 정점입니다.
- 루트를 제외한 모든 정점은 결국 내부 정점의 자식입니다.

2. 트리의 정리

- n 개의 정점을 가진 트리는 $n-1$ 개의 모서리를 가집니다.
- n 개의 내부 정점을 가진 포화 m 가지 트리는 $m*n + 1$ 개의 정점이 있습니다. 내부 정점의 정의를 읽어보면 내부 정점은 모두 m 개의 자식이 존재하고 루트는 세지 않으므로 1을 더해줍니다.
- 루트의 depth는 0인 걸 제대로 명시해 줍니다. 교재에서는 depth를 레벨로 표기했습니다.

3. 트리의 종류

1) 루트 트리

= 루트에서 지정된 한 개의 정점과 모든 모서리들이 루트로부터 나가는 방향을 갖는 트리.

2) 포화 m가지 트리

= Leaf를 제외하고 모든 정점이 동일하게 m개의 자식을 갖는 트리

- 대표적으로 포화 이진트리($m=2$)

- 예제 3 참고

3) 균형 m가지 트리

= 모든 리프의 depth가 (트리의 depth) 또는 (트리의 depth - 1)인 트리

- 균형 m가지 트리는 검색이 빠릅니다.

4) 순서 루트 트리

= Left 자식 , Right 자식이 존재하는 가장 일반적으로 우리가 사용하는 트리

4. 트리일 조건

- 연결 그래프

- 사이클 X

- $n-1$ 개의 edge를 가집니다.

5. 문제 풀이

<예제 1>

- 모두 트리가 아닙니다.

<예제 3>

- 포화 m가지 트리
- 언제나 leaf는 제외하고 고려해야 합니다.
- T1 : 포화 이진트리
- T2 : 포화 3가지 트리
- T3 : 포화 5가지 트리
- T4: 자식의 개수가 2 OR 3 이므로 포화 m가지 트리 X

<예제 10>

- 트리의 높이 : 4

<예제 11>

- T1 , T3 : 균형 루트트리
- T2는 depth가 2인 리프가 존재하므로 균형 루트 트리 X

<연습문제 12>

- 네 개의 정점을 갖는 비동형 비루트 트리 : 2개



-
- 네 개의 정점을 갖는 비동형 루트 트리 : 2개



<연습문제 40 & 42>

① 비루트 트리의 편심률

= 어떤 정점에서의 모든 단순 경로의 길이 중 최소 길이

ex) 정점_e는 최대 경로의 길이가 3

② 중심

= 트리에서 어떤 정점 보다 작은 편심률을 갖는 정점이 존재하지 않는다면 그 정점을 중심이라 합니다.

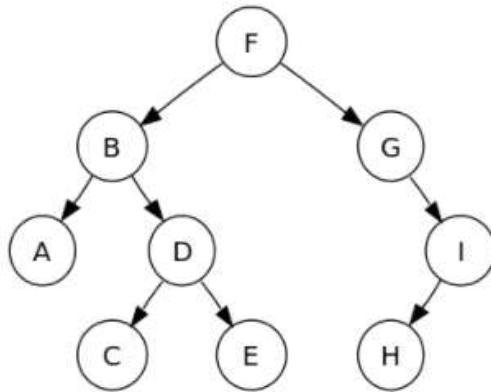
- 비루트 트리에서 최소 높이의 루트 트리를 생성하려면 중심을 루트로 선택해야 합니다.

ex) 정점_e의 편심률이 가장 작기 때문에 정점_e가 비루트 트리에서 중심입니다.

<11-3강_트리 순회> 12/04

1. 탐색 알고리즘

- 방문(Visit) : 순회의 순서를 적을 때 visit을 완료한 순서로 적습니다.
- 방문의 순서에 따라 전위 , 중위 , 후위로 나눕니다.



이진 탐색 트리에서

- 전위 순회: F, B, A, D, C, E, G, I, H (root, left, right)
- 중위 순회: A, B, C, D, E, F, G, H, I (left, root, right)
- 후위 순회: A, C, E, D, B, H, I, G, F (left, right, root)
- 레벨 순서 순회: F, B, G, A, D, I, C, E, H

*** 3가지 순회 모두 예시를 보면서 따라하면 매우 간단합니다. ***

1) 전위 순회 (Visit -> Left -> Right)

- Visit , Left , Right를 노드별로 차례대로 수행합니다.
- F에서 visit 하고 left가 가능한지 판별합니다. left가 가능하므로 B로 이동하고 다시 (Visit -> Left -> Right)를 수행합니다.
- B에서도 가능하므로 A로 이동합니다. A는 visit만 되고 left , right가 불가능하므로 B로 거슬러 올라갑니다. B에서는 Visit , Left를 했으므로 Right를 수행합니다.
- D에서 Visit하고 Left를 합니다.

2) 중위 순회 (Left -> Visit -> Right)

3) 후위 순회 (Left -> Right -> Visit)

2. 수식의 표기법

- 우리가 배운 수식은 중위 표기법으로 나타낼 수 있습니다.

1) 전위 표기법

- 연산자를 만나면 연산자 기준 오른쪽에 있는 2개의 수를 연산합니다.

전위 표기법 $+ - * 2 3 5 / \uparrow 2 3 4$ 의 값을 구하라.

$$\begin{array}{cccccccc} + & - & * & 2 & 3 & 5 & / & \uparrow & 2 & 3 & 4 \\ & & & & & & & \underbrace{} & & & \end{array}$$

$2 \uparrow 3 = 8$

$$\begin{array}{cccccccc} + & - & * & 2 & 3 & 5 & / & 8 & 4 \\ & & & & & & \underbrace{} & & \end{array}$$

$8 / 4 = 2$

$$\begin{array}{ccccccc} + & - & * & 2 & 3 & 5 & 2 \\ & & \underbrace{} & & & & \end{array}$$

$2 * 3 = 6$

$$\begin{array}{ccccccc} + & - & 6 & 5 & 2 \\ & \underbrace{} & & & \end{array}$$

$6 - 5 = 1$

$$\begin{array}{ccc} + & 1 & 2 \\ \underbrace{} & & \end{array}$$

$1 + 2 = 3$

① $2 \wedge 3 = 8$

② $8 / 4 = 2$

③ $2 * 3 = 6$

④ $6 - 5 = 1$

⑤ $1 + 2 = 3$

2) 후위 표기법

- 연산자를 만나면 연산자 기준 왼쪽에 있는 2개의 수를 연산합니다.

후위 표기 $723^*-4^{93}/+$ 의 값을 구하라.

[illegible]

① $2 * 3 = 6$

② $7 - 6 = 1$

③ $1 \wedge 4 = 1$

④ $9 \div 3 = 3$

⑤ $1 + 3 = 4$

3. 문제 풀이

<예제 1>

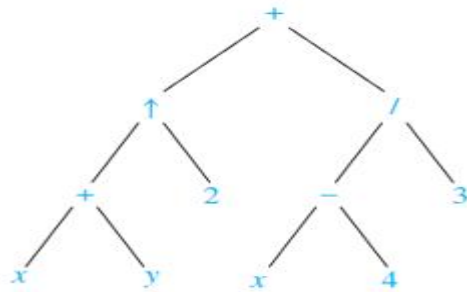
- 트리를 왼쪽부터 방문하는 방식.

<예제 5 & 예제 6 & 예제 8>

- 예제 5에서 중위 수식에 대한 순서 루트 트리를 구합시다. 그 트리를 전위 순회하면 예제 6의 정답이고 , 후위 순회하면 예제 8의 정답입니다.

① 중위 순회 트리

- 리프에는 숫자가 들어가고 , 내부 정점에는 연산자가 들어갑니다.
- 괄호 안에 있을수록 depth가 낮아집니다.



② 전위 순회 트리

= 전위 순회로 나타내면 됩니다.

= $^{+}+xy2/-x43$

③ 후위 순회 트리

= 후위 순회로 나타내면 됩니다.

= $xy+2^{x}4-3/+$

<연습문제 30>

- 잘 정의된 연산 : 연산자 개수 + 1 = 변수의 개수
- a) : 연산자 3개 , 변수 3개 이므로 , 연산자가 1개 남습니다.
- b) : 연산자 2개 , 변수 4개 이므로 , 연산자가 1개 부족합니다.
- c) : 연산자 4개 , 변수 4개 이므로 , 연산자가 1개 남습니다.
- d) : 동그라미는 연산자 입니다. 연산자 4개 , 변수 5개 이므로 잘 정의 되었습니다.

<연습문제 34>

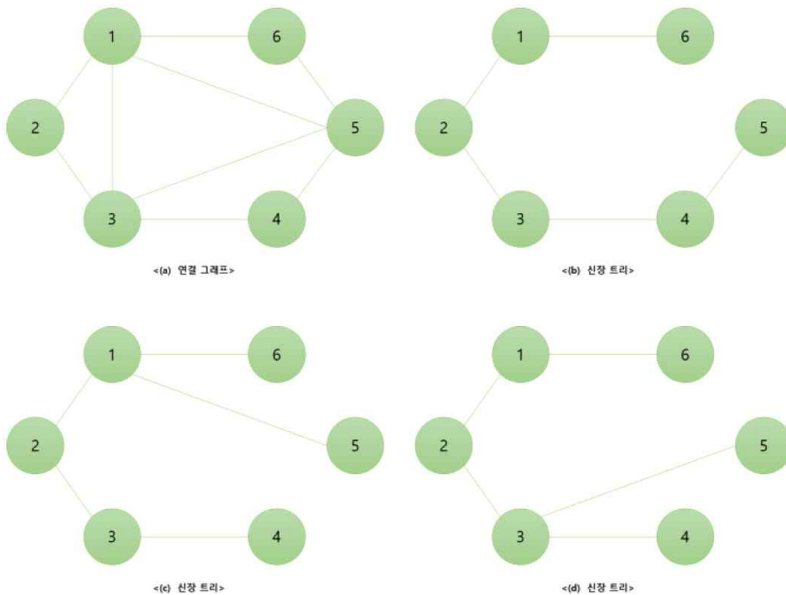
- 잘 정의된 연산을 필자가 바꾼다면 그에 맞게 찾으려 합니다.

<11-4강_신장 트리> 12/05

1. 신장 트리(Spanning Tree)

- = 연결 그래프(임의의 두 노드를 선택했을 때 항상 경로가 존재)이며 , 사이클이 없는 트리.
- 단순 그래프가 신장 트리를 가지면 그 단순 그래프는 연결되어 있습니다.
- 단순 연결 그래프에서 반드시 신장 트리가 존재하는 이유 : 단순 순환 중에서 한 개에서 한 모서리를 제거
- 모두 연결되어 있고 사이클이 없는 신장 트리야 말로 , 탐색 알고리즘을 수행하기 가장 최적화 된 트리라고 생각이 듭니다.(개인적인 생각)
- 연결 그래프의 정의가 헛갈리면 10-4강에서 복습하고 옵시다.

<Example : 연결 그래프를 이용해서 다양한 신장 트리를 만들어 봅시다. -> 사이클을 제거합니다. >



신장 트리 조건

- ① 기존의 연결 그래프의 모든 정점을 포함해야 합니다.
 - ② 신장 트리도 연결 그래프여야 합니다.
- = 임의의 두 노드를 선택했을 때 항상 경로가 존재합니다.
- ③ 트리어므로 사이클이 존재하지 않아야 합니다.

2. DFS & BFS

- DFS , BFS 관련 내용은 많이 공부했기 때문에 따로 필기하지 않았습니다.

3. 문제 풀이

<예제 1>

- 원래 그래프에서 연결 그래프의 성질은 유지하고 모든 정점은 갖는데 몇 개의 edge를 지운다로 생각해도 됩니다.

<예제 3>

- 그래프를 DFS를 통해 신장 트리를 그립니다.

<예제 5>

- 그래프를 BFS를 통해 신장 트리를 그립니다.

<예제 7>

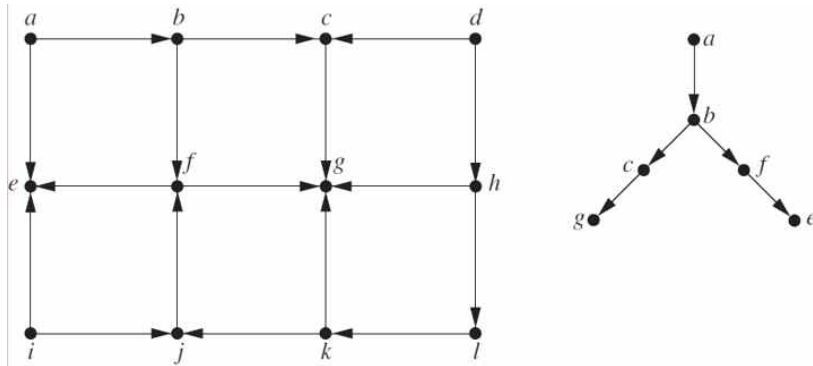
- N-Queen 문제

<예제 8>

- DFS로 원소들의 합이 39인 경우를 찾습니다.

<예제 9>

- 비방향성 그래프가 (약한)연결 그래프여서 신장 트리를 만들 수 있었습니다. 이 비방향성 그래프에 임의로 방향을 추가하여 방향성 그래프를 만들었을 때 언제나 신장트리를 만들 수는 없습니다. 방향성 그래프를 만든다고 해서 강한 연결성을 갖는 다는 보장이 없으므로 임의의 두 점을 선택했을 때 항상 경로가 존재하지 않을 수 있습니다.



- a에서 g로 가는 경로는 존재하지만 , g에서 a로 가는 경로는 존재하지 않습니다.

<예제 10>

- Google의 검색엔진 탐색은 방향성 그래프로 생각할 수 있으며 depth를 제한한 탐색을 합니다.

<연습문제 1>

- 연결 그래프는 그래피므로 사이클이 존재할 수 있지만 신장 트리는 사이클이 존재하면 되지 않기 때문에 사이클이 존재하는 경우 $m - n + 1$ 개의 edge를 제거해야 합니다.
- 사이클이 없는 연결 그래프는 edge를 제거하지 않아도 신장 트리를 만들 수 있습니다.

<연습문제 8 & 10>

- 신장 트리의 정의를 생각하면 쉽게 구할 수 있습니다.

<연습문제 12>

- 형태가 다른(비동형) 신장트리
- k3 : 1가지
- k4 : 2가지(돌리거나 뒤집으면 같은 것입니다.)
- k5 : 3가지

<연습문제 22>

- Q_n 에서의 BFS로 만든 신장트리를 2개 합치면 Q_{n+1} 의 신장트리가 됩니다.
- Q_n 에서의 DFS는 일직선으로 연결하면 됩니다.

<연습문제 24>

- DFS 또는 BFS를 통해 탐색하는 알고리즘에서 순서를 출력하는 방법 : visit할 때 마다 출력

<연습문제 30>

- DFS와 백트래킹을 통해 미로에서 출구를 찾는 방법 : 일단 가다가 막히면 다시 돌아오고 , 이전 depth에서 다른 노드로 가봄을 반복.

<연습문제 42>

- BFS

<11-5강_최소 신장 트리> 12/06

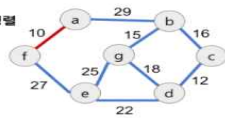
1. 최소 신장 트리 (Minimum Spanning Tree)

- = 연결 가중치 그래프에서 모서리의 가중치 합이 최소인 신장트리
- 가중치가 있는 그래프에서 고려하는 개념입니다.

2. Kruskal Algorithm

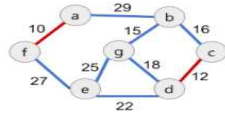
- = Cycle(단순 순환)을 형성하지 않고 가중치가 적은 순서로 Edge를 선택하여 연결합니다.

1) 간선들의 가중치 오름차순 정렬



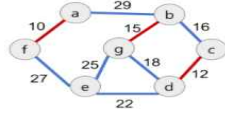
af	cd	bg	bc	dg	de	eg	ef	ab
10	12	15	16	18	22	25	27	29

2)



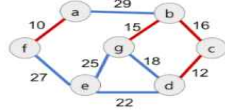
af	cd	bg	bc	dg	de	eg	ef	ab
10	12	15	16	18	22	25	27	29

3)



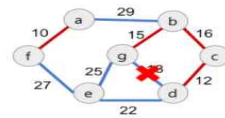
af	cd	bg	bc	dg	de	eg	ef	ab
10	12	15	16	18	22	25	27	29

4)



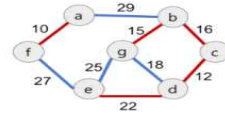
af	cd	bg	bc	dg	de	eg	ef	ab
10	12	15	16	18	22	25	27	29

5) 사이클 형성. dg는 제외



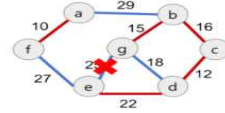
af	cd	bg	bc	dg	de	eg	ef	ab
10	12	15	16	18	22	25	27	29

6)



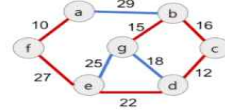
af	cd	bg	bc	dg	de	eg	ef	ab
10	12	15	16	18	22	25	27	29

7) 사이클 형성. eg는 제외



af	cd	bg	bc	dg	de	eg	ef	ab
10	12	15	16	18	22	25	27	29

8) N-1개의 간선 생성. 종료



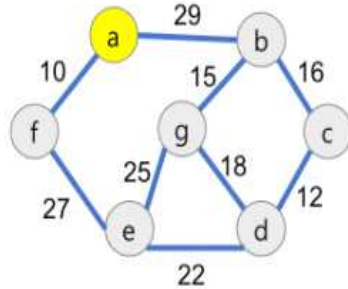
af	cd	bg	bc	dg	de	eg	ef	ab
10	12	15	16	18	22	25	27	29

3. Prim Algorithm

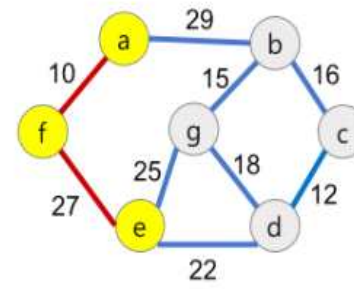
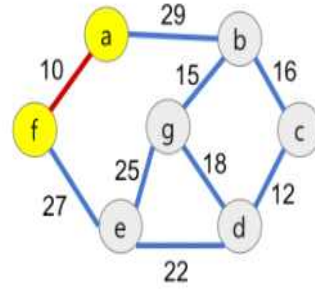
= 정점의 집합과 인접한 edge 중에서 cycle을 형성하지 않고 최소 가중치를 갖는 edge를 선택하여 연결합니다.

- 하나의 정점과 인접한 edge 중에서 선택하는 것이 아니라, 지금까지 방문한 정점들의 집합과 인접한 edge 중에서 선택해야 합니다.

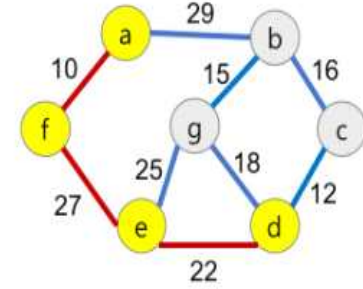
1) 시작 단계: 시작 정점만 포함



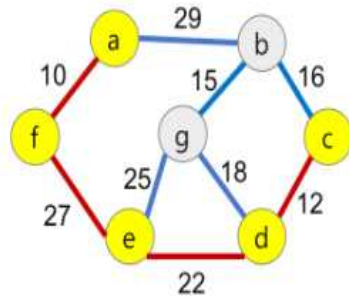
2) 인접 정점 중 최소 간선으로 연결된 정점 선택 3)



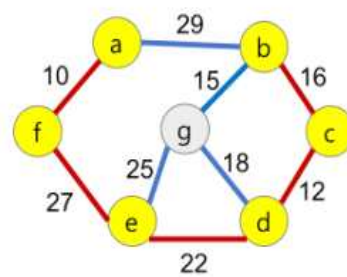
4)



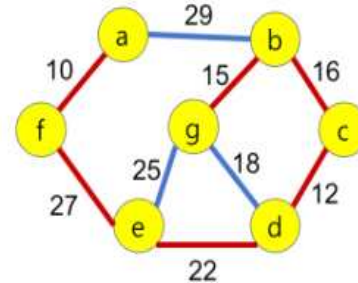
5)



6)



7) N-1개의 간선 생성. 종료



4. Kruskal vs Prim

① Kruskal

- Edge를 가중치의 크기별로 정렬을 하면 되므로, 정렬 알고리즘과 가장 연관이 있습니다.
- 좋지 않은 복잡도를 가진 정렬 알고리즘(구현하기 쉬움)을 사용하면 $O(E^2)$ 의 복잡도로 계산이 가능합니다. -> 비추
- STL을 사용하면 kruskal을 가장 빠르게 계산할 수 있는 Quick-Sort를 사용하기 때문에 $O(E * \log E)$ 의 복잡도로 계산이 가능합니다.
- Edge의 개수가 Node의 개수 보다 적은 경우 사용합시다.

② Prim

- 우선순위 큐를 사용해서 방문한 모든 노드와 인접한 노드와의 edge를 저장하는 자료구조.
- 좋지 않은 복잡도를 가진 정렬 알고리즘(구현하기 쉬움)을 사용하면 $O(V^2)$ 의 복잡도로 계산이 가능합니다. -> 비추
- 우선순위 큐를 사용하면 $O(E * \log V)$ 의 복잡도로 계산이 가능합니다.
- Node의 개수가 Edge의 개수 보다 적은 경우 사용합시다.

5. 문제 풀이

<예제 2>

- Prim Algorithm 사용

<연습문제 2>

- Prim Algorithm 사용

<연습문제 6>

- Kruskal Algorithm 사용

<연습문제 11 & 12>

- Maximum Spanning Tree
- 최소를 찾지 않고 최대를 찾으려면 되는 간단한 문제입니다.

<연습문제 18>

- 연결 가중치 그래프에서 최소 가중치를 갖는 모서리들은 신장 트리의 일부분입니다.

<연습문제 22>

- MST에서 특정 edge는 무조건 포함해야 합니다.
- kruskal로 생각하면 탐색을 시작하기 전에 문제에서 주어진 edge를 모두 추가하고 진행합니다. 문제에서 주어진 edge를 추가했을 때 cycle이 형성 된다면 MST를 구현할 수 없습니다.

<연습문제 34>

- **역 제거 알고리즘** : MST에서 최소 비용을 찾을 때 , 그래프에서 연결성을 깨뜨리지 않는다는 조건하에 최대 가중치를 갖는 edge들을 제거.